Institut für Maschinelle Sprachverarbeitung
Universität Stuttgart
Pfaffenwaldring 5B
D-70569 Stuttgart

Master Thesis

Interest Representations in Deep News Recommender Systems

Aditi Godbole

Course:	M.Sc. Computer Science
Examiner:	Prof. Dr. Sebastian Padó
	Dr. Agnieszka Faleńska
Supervisor:	Lucas Möller
Start of Thesis:	15.03.2024
End of Thesis:	15.09.2024

Abstract

News recommender systems (NRS) aim to reduce information overload by suggesting articles tailored to user interests. However, traditional systems often rely on single-vector user representations. These may fail to capture the full diversity of user preferences. This thesis introduces a novel approach using multi-interest user representations which is combined with disentanglement objective to ensure distinct and non-overlapping interest vectors. The study includes a review of both traditional and deep learning-based news recommendation methods, followed by the development of new multi-interest model. This model is tested on the MIND dataset, a large collection of user behavior logs from Microsoft News. The evaluation focuses on enhancing click prediction accuracy, achieving clear and separate interest representations, improving recommendation fairness and diversity, and reducing bias through geometric analysis of embeddings.

Results demonstrate that multi-interest user representations enhance click prediction accuracy and produce more balanced recommendations. Disentanglement techniques reduce overlap between interest vectors, creating clearer user profiles. However, the approach may reduce recommendation diversity, suggesting a need for careful tuning. This framework offers the potential for more personalized, fair, and unbiased news recommendations across various domains.

Acknowledgements

I express my deepest gratitude to my supervisor, Lucas Möller, for his continuous support, guidance, and invaluable insights throughout this research. His expertise and encouragement have been instrumental in the completion of this thesis. I am also deeply indebted to Prof. Dr. Sebastian Padó, whose feedback and suggestions have greatly improved the quality of my work. Lastly, I would like to thank my family and friends for their unwavering support and belief in me. Their love and encouragement have been my driving force throughout this journey.

Contents

List of Figures

List of Tables

List of Abbreviations

- **DL:** Deep Learning
- **ML:** Machine Learning
- **NLP:** Natural Language Processing
- **RS:** Recommender Systems
- **BERT:** Bidirectional Encoder Representations From Transformers
- **GPT:** Generative Pre-trained Transformers
- **NRS:** News Recommender Systems
- **NPA:** Neural News Recommendation with Personalized Attention
- **NAML:** Neural News Recommendation with Attentive Multi-View Learning
- **NRMS:** Neural News Recommendation with Multi-Head Self-Attention
- **LSTUR:** Long- and Short-term User Representations
- **CAUM:** Candidate-Aware User Modeling
- **MINS:** Multi-Interest News Sequence Modeling

Chapter 1

Introduction

1.1 Background of the study

The newspaper industry has changed significantly over the past two decades. Today, readers can choose from various news sources, such as traditional newspaper websites, digital-only platforms, and news aggregation services like Google and Yahoo. The digital format allows publishers to update content in real time. This has greatly accelerated the publication process. According to the Reuters Institute Digital News Report (2016) Levy et al. (2017), real-time data has led to a steady increase in online users. However, the sheer volume of constantly updated information creates a problem. Readers often find it difficult to focus on the most relevant news, leading to confusion and making it hard to understand the content (Karimi, 2018)Karimi et al. (2018).

In situations like this, tools and technologies such as news recommender systems (NRS) are essential. They help users navigate information in a more refined way. Recent advancements in information technology have made it possible for people to access

a vast array of news through various data sources and news aggregators like Google News. These tools facilitate the discovery of relevant and authentic content. They offer users personalized recommendations based on their interests and search history Talha et al. (2023). Recommender systems have shown great potential in addressing the issue of information overload. These systems filter information streams according to user preferences and suggest additional relevant items. There have been substantial advancements in recommendation technology across various domains, including movies, books, travel services, research articles, and more Jannach et al. (2010); Beel et al. (2016); Borràs et al. (2014); Park et al. (2012). NRS reduces the burden of manual searching by quickly and efficiently delivering news tailored to users' preferences. These systems create new suggestions for articles and news items by analyzing users' past behaviour and interests.

Recommending appropriate news is challenging due to the dynamic nature of the news domain and the diverse interests of users. This presents unique challenges compared to other applications of recommender systems Raza and Ding (2022). Users might be interested in a wide range of topics, and their interests can change over time. An et al. (2019) emphasizes that a key problem in news recommendation is learning accurate user representations to capture these changing interests. Wu et al. (2019b) further notes that different users usually have different interests, and even the same user may have varying interests at different times. To manage the continuous influx of information, NRS use user interest models based on browsing history. This enables users to quickly and precisely find news or articles that match their interests. This personalized approach ensures that users receive the most relevant updates without needing explicit requirements Raza and Ding (2022). By analyzing past behavior and preferences, NRS provide tailored recommendations that adapt to the changing interests of users. This helps them navigate the

vast amount of information available online efficiently.

In a recent study on news recommenders at the Institute for Natural Language Processing (IMS), University of Stuttgart, researchers uncovered intriguing patterns in user behaviour Möller and Padó (2024). They found that users tend to cluster closely within the model's embedding space, while news articles are more widely distributed. This configuration resulted in a noticeable bias. Certain types of news consistently received high recommendations across most users, while other types were rarely suggested. Understanding the origins of this bias remains a central topic of interest in the field.

Problem Statement:

News recommender systems often use single-vector user representations computed from users' click history. This method creates a one-dimensional vector reflecting past interactions, but it has a key limitation. **Lack of Diverse Interest Representation** - Single-vector representations may struggle to capture the full range of a user's interests. For instance, a user interested in politics, sports, and technology may not be adequately represented by a single vector.

To improve the relevance and accuracy of news recommendations, this research proposes using **multi-interest user representations**. These representations consist of multiple vectors instead of a single one, with each vector intended to represent a different aspect of the user's interests. A disentanglement objective will be included to ensure different interests are represented without overlap. The effectiveness of these multi-interest representations will be compared with traditional single-vector models to evaluate their impact on recommendation quality.

1.2 Research Objectives and Questions

This thesis aims to develop a novel user representation model for news recommender systems by addressing the following key questions:

1. **Enhancing Click Prediction Accuracy:** Can multi-interest user representations improve click prediction accuracy and the relevance of recommended news articles compared to single-vector representations?

2. **Clear Differentiation of User Interests:** How can we distinctly separate and represent different user interests, reducing overlap and ensuring clarity?

3. **Improving Fairness, Diversity, and Reducing Bias:** How do multi-interest user representations impact recommendation fairness and diversity? Can they reduce systemic biases and lead to broader content coverage?

1.3 Scope of the Study

This study aims to develop and evaluate a multi-interest user representation model for news recommender systems, targeting improved accuracy, fairness, and adaptability. Using the Microsoft News Dataset (MIND), the model will incorporate a disentanglement objective to better capture diverse user interests. Key metrics include click prediction accuracy, user interest separation and recommendation fairness and diversity.The proposed model will be compared to traditional single-vector representations. The research is focused specifically on news recommender systems, addressing the challenge of delivering a more personalized and fair news recommendation experience.

1.4 Structure of the Thesis

The thesis is organized as follows:

- **Chapter 1: Introduction** Outlines the research background, problem statement, objectives, and thesis structure.
- **Chapter 2: Theoretical Background** Discusses key theoretical concepts like transformers and news recommender systems.
- **Chapter 3: Literature Review** Summarizes existing research on news recommender systems, identifying gaps this study addresses.
- **Chapter 4: Methodology** Explains the proposed model and discusses assumptions and limitations.
- **Chapters 5: Experiments, Results and Discussion** Details about the data and experiments, including setup, evaluation, results, and discussion.
- **Chapter 6: Conclusion and Outlook** Summarizes findings, discusses implications, and suggests future research directions.

Chapter 2

Theoretical Background

2.1 Introduction to Theoretical Background

This chapter covers the theoretical foundations of machine learning (ML) and deep learning (DL) as applied to news recommender systems. We will explore deep learning, transformer architecture, dual encoder models, and the evolution of news recommendation techniques. We will highlight key advancements and their relevance to personalized recommendations.

2.2 Deep Learning

2.2.1 Introduction to Deep Learning

Deep learning (DL), a subset of machine learning (ML), has gained popularity for its ability to automatically model complex patterns in data without manual feature engineering.

DL models can automatically learn hierarchical representations of data through multiple layers of abstraction. This makes DL particularly impactful in domains like natural language processing (NLP), computer vision (CV), and recommender systems (RS), where large amounts of unstructured data are commonLeCun et al. (2015). DL models, inspired by the human brain's neural architecture, consist of interconnected layers that uncover intricate patterns by leveraging vast data and computational power. This ability to capture both low-level and high-level features within the data allows DL to excel in tasks like image recognition, speech processing, and personalized recommendations Schmidhuber (2015).

2.2.2 Key Components of Deep Learning Models

In the following paragraphs, we take a look at the key components of DL models.

Input Layer: The input layer is the first layer of a DL model. It is responsible for receiving and processing raw data. In the context of recommender systems, this data could include user interaction histories (such as clicks, ratings, and browsing behavior), textual content from news articles, user profiles, and other relevant features. The primary function of the input layer is to prepare this raw data for further processing by the hidden layers of the network. For instance, in a news recommender system (NRS), the input might consist of the textual content of articles that users have interacted with. It includes metadata such as publication dates, and user-specific data LeCun et al. (2015). In DL, the data fed into the input layer is often preprocessed and normalized to ensure that the model can efficiently learn from it. For textual data, techniques such as word embeddings (e.g., Word2Vec, GloVe) are commonly used to convert words into dense vectors. These dense vectors capture semantic meaning which the model can then process Mikolov (2013).

Hidden Layers: Hidden layers are the core components of a DL model and are positioned between the input and output layers. Each hidden layer performs a series of transformations on the data it receives. It typically involves linear operations followed by non-linear activation functions. These transformations enable the model to learn abstract representations of the data, which are critical for capturing complex patterns and relationships. The depth of a neural network, defined by the number of hidden layers, allows the model to learn from simple features in the initial layers to more complex features in the deeper layers. For example, in image processing, early layers might detect edges and textures, while deeper layers recognize objects and scenes. In RS, hidden layers help model user preferences and item features, which ultimately enable the system to make personalized recommendations Goodfellow (2016). In the context of news recommendations, hidden layers might process the semantic content of articles and user interaction patterns to identify the underlying factors that influence user preferences.

Output Layer: The output layer is the final layer of the DL model. It is responsible for generating predictions or classifications based on the processed data from the hidden layers. In recommender systems, the output layer might produce predictions such as the likelihood of a user clicking on a particular article, the relevance score of an item to a user, or the ranking of items based on predicted user interest. The structure of the output layer varies based on the task. For binary classification, a single neuron with a sigmoid activation outputs a probability, while a softmax layer is used for multi-class classification to produce a probability distribution. In recommender systems, the output could be a ranked list of items or relevance scores, indicating how well each item matches the user's preferences Covington et al. (2016). In a news recommender system, this might translate to a list of articles ranked by predicted relevance or a probability score for user engagement,

helping tailor the user's experience Goodfellow (2016).

2.2.3 Training Deep Learning Models

Training deep learning models involves adjusting model parameters to minimize the difference between predictions and actual outcomes through a process called backpropagation Rumelhart et al. (1986). During training, the model's predictions are compared to actual results (e.g., user clicks), and the error, or loss which is calculated using a loss function. Backpropagation computes the gradient of this loss with respect to the model's weights and updates them to reduce the error.

Optimizers like Stochastic Gradient Descent (SGD) and Adam are used to adjust weights during training, helping the model converge Kingma (2014). Techniques such as dropout and batch normalization prevent overfitting, while regularization methods like L2 regularization penalize large weights, ensuring the model generalizes well to new data Srivastava et al. (2014). This is crucial for recommender systems (RS) to generalize user preferences beyond the training set.

2.3 Transformers and Embeddings

Overview of Transformers

Transformers have become a dominant architecture in natural language processing (NLP) due to their ability to model long-range dependencies and context within text. Their influence extends to recommender systems, where they generate high-quality embeddings that capture semantic meaning and contextual nuances. Central to this is the multi-head

self-attention mechanism, which allows the model to focus on different parts of a sentence simultaneously, capturing diverse relationships between words. Positional encoding complements this by ensuring the model understands word order, adding position-specific vectors to each word embedding (Vaswani, 2017; Gehring et al., 2017).

The processed data then passes through feedforward neural networks, which apply non-linear transformations to capture complex patterns. Techniques like layer normalization and residual connections stabilize the training process by maintaining consistent outputs and preventing degradation of information through multiple layers (Ba et al., 2016; He et al., 2016). Transformers are pre-trained on large corpora using tasks like masked language modeling (BERT) or autoregressive prediction (GPT), enabling them to learn rich, context-sensitive embeddings. These embeddings can be fine-tuned for specific tasks, such as recommender systems, improving recommendation accuracy and personalization (Devlin, 2018; Radford et al., 2018).

2.3.1 Generating Embeddings with Transformers

Understanding Embeddings: Embeddings are dense vector representations of text that capture the semantic meaning and contextual relationships of words or sentences. Transformers, particularly models like BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pre-trained Transformer), have set the standard for generating high-quality embeddings used in a wide range of downstream tasks, including recommender systems.

BERT (Bidirectional Encoder Representations from Transformers): BERT, introduced by Devlin (2018), is a pre-trained transformer model designed to capture the bidi-

rectional context of words. Unlike traditional models that process text in one direction, BERT considers both left and right contexts simultaneously, producing more contextually rich embeddings. It is trained using a masked language modeling (MLM) objective, where some tokens are masked, and the model predicts them, learning deep contextual representations (Devlin, 2018).After pre-training, BERT can be fine-tuned for specific tasks like news or product recommendation, where the model's parameters are adjusted to optimize task-specific performance. This fine-tuning process tailors BERT's embeddings to the particular domain, improving recommendation accuracy (Sun et al., 2019).

GPT (Generative Pre-trained Transformer): GPT, introduced by Radford et al. (2018), generates embeddings using an autoregressive modeling technique, predicting the next token based on previous tokens. This approach is useful for tasks like text generation and completion, and in recommender systems, it helps model user interaction sequences, improving predictions of user behavior (Radford et al., 2018).Like BERT, GPT is pre-trained on large text corpora and can be fine-tuned for specific tasks. However, GPT's focus is on generating coherent sequences, making it ideal for tasks such as dialogue generation and content recommendation, where understanding language structure is key (Brown, 2020).

2.4 Dual Encoder Models

2.4.1 Introduction to Dual Encoder Architecture

Dual encoder architectures map two types of inputs, such as users and items, into a shared embedding space using dedicated encoders. Then, they compare the resulting embeddings

using a distance function to assess their compatibility. This enables personalized and accurate recommendations.

2.4.2 Components of Dual Encoder Models

Encoder 1: The first encoder processes input data from one modality or domain, transforming it into a dense vector representation (embedding). This could involve text, images, or other structured or unstructured data forms. For text-based inputs, transformer-based models like BERT (Devlin, 2018) or GPT (Radford et al., 2019) are often employed, leveraging their ability to capture bidirectional context and semantic richness. In vision-related tasks, convolutional neural networks (CNNs) (Krizhevsky et al., 2012) or Vision Transformers (ViT) (Dosovitskiy, 2020) are commonly used to encode images.

Encoder 2: The second encoder functions similarly to Encoder 1 but operates on a different input modality or domain. For example, in the vision-language model CLIP (Radford et al., 2021), Encoder 1 processes images, while Encoder 2 processes corresponding text captions. The goal of Encoder 2 is to generate embeddings that represent its input data in a space where they can be meaningfully compared with the embeddings from Encoder 1.

Similarity Function: Once both encoders generate embeddings, a similarity function is employed to compare the embeddings and determine their relevance or match. Popular similarity measures include cosine similarity, Euclidean distance, and dot product (Gillick et al., 2018). These measures allow the model to score how closely related the inputs are, which can be used for retrieval, ranking, or other downstream tasks.

2.4.3 Working Mechanism

The functioning of dual encoder models generally involves the following steps:

1. **Data Processing:** User and item data are preprocessed to extract relevant features. This may include tokenization for textual data, normalization for numerical data, and encoding for categorical data.

2. **Embedding Generation:** The preprocessed data is fed into the respective encoders—one for the user and one for the item. These encoders which are often based on transformers, generate high-dimensional embeddings that represent the user and item in a latent space. Transformers excel at capturing complex relationships and contextual information, which are encoded into these embeddings.

3. **Similarity Calculation:** The generated embeddings are passed through a similarity function that calculates the degree of match between the user and item embeddings. The result is a relevance score that indicates how likely the user is to interact with the item.

4. **Ranking and Recommendation:** Based on the similarity scores, items are ranked. The top-ranked items are then presented to the user as recommendations. The efficiency of dual encoder models allows for real-time recommendations as embeddings can be precomputed and stored thereby enabling quick retrieval during inference (Humeau et al., 2019).

2.4.4 Applications in Recommender Systems

Dual encoder models provide significant advantages in large-scale recommender systems by allowing the precomputation of embeddings for both users and items, enabling efficient real-time recommendations. In search engines, dual encoders are used to match user queries with relevant documents or web pages, facilitating fast and scalable search operations (Gillick et al., 2018). In conversational AI, these models help dialogue systems by matching user queries with appropriate responses, generating embeddings for both user inputs and potential replies to select the most contextually suitable response (Henderson et al., 2019). Platforms like YouTube and Netflix utilize dual encoders in content recommendation, where they match users with videos or shows based on viewing history and preferences. The transformer-based embeddings capture the rich semantic content of the media, resulting in more accurate recommendations (Covington et al., 2016).

2.5 News Recommender Systems

2.5.1 Introduction to News Recommender Systems

News recommender systems (NRS) have evolved from basic rule-based methods to sophisticated models that leverage deep learning and hybrid techniques. These advancements address the challenge of managing and personalizing the vast amount of online news content, ensuring users receive articles aligned with their interests. The primary goal of NRS is to reduce information overload and enhance user experience through personalized recommendations, a crucial aspect in the dynamic and rapidly changing news environment. Developing models that are both accurate and adaptive to evolving user

preferences remains a significant challenge, requiring sophisticated algorithms capable of handling this complexity (Karimi et al., 2018). This section provides an overview of the development of NRS, exploring their architecture and popular models used in the field.

2.5.2 Evolution of Methods in News Recommender Systems

The evolution of NRS can be categorized into three main phases: traditional methods, deep learning-based methods, and hybrid approaches.

Traditional Methods

Traditional news recommendation methods have established the groundwork for more sophisticated techniques by leveraging user-item interactions and content analysis. The various approaches are discussed in the following paragraphs.

Collaborative Filtering (CF) is one of the earliest and most widely used techniques in recommender systems. It operates on the principle that users with similar past preferences are likely to enjoy the same items in the future. CF is typically divided into memory-based and model-based approaches. Memory-based CF uses user-item interaction matrices to find similarities among users or items, while model-based CF leverages machine learning techniques like matrix factorization to predict user preferences (Sarwar et al., 2001; Koren et al., 2009). Despite its effectiveness, CF faces challenges such as data sparsity and the cold start problem (Schafer et al., 2007).

Content-Based Filtering, on the other hand, recommends articles by analyzing their textual content and matching them with user profiles generated from reading history. Techniques like Term Frequency-Inverse Document Frequency (TF-IDF) and Latent Se-

mantic Indexing (LSI) are commonly used to represent textual content and calculate similarities between user profiles and articles (Pazzani and Billsus, 2007; Lops et al., 2011). However, this approach can lead to the "filter bubble" effect, where users are only exposed to content that reinforces their existing views, limiting diversity in perspectives.

Hybrid Methods combine the strengths of both collaborative filtering and content-based approaches to enhance recommendation accuracy and mitigate the limitations of each technique. For instance, Melville et al. (2002) proposed a hybrid model that incorporates content-based features into collaborative filtering, effectively addressing challenges like the cold start problem and data sparsity.

Deep Learning Methods

The introduction of deep learning has revolutionized news recommendation by enabling the modelling of complex user-item interactions and automatically learning rich feature representations from raw data.

Neural networks, including Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), are widely used in deep learning-based news recommender systems (NRS). CNNs, originally for image processing, extract meaningful patterns from news content (Kim, 2014), while RNNs, particularly Long Short-Term Memory (LSTM) networks, model sequential data to capture user behavior and predict future interests (Hochreiter and Schmidhuber, 1997). Attention mechanisms, like self-attention in Transformer models, improve context-aware recommendations by assigning varying importance to different inputs (Vaswani, 2017). Transformer-based models such as BERT and GPT have set new standards in NRS. BERT's bidirectional context enhances representations of news articles and user preferences (Devlin, 2018), and GPT's ability to generate

coherent sequences helps predict the next content a user might engage with, improving personalization and accuracy (Brown, 2020).

Hybrid Deep Learning Models

Hybrid models that combine deep learning with traditional recommendation techniques have shown great promise in NRS.

Neural Collaborative Filtering (NCF) integrates deep learning with traditional collaborative filtering methods to enhance recommendation accuracy. By combining matrix factorization with neural networks, NCF models user-item interactions and captures non-linear relationships between users and items. This integration allows for more sophisticated modeling of preferences, leading to improved recommendations (He et al., 2017).

Representation aggregation is another approach used in hybrid models like LSTUR and MINS, which combine representations from various sources, including short-term and long-term user behaviors. By aggregating these representations, the models create comprehensive user profiles that capture both immediate interests and long-term preferences. This method leads to more accurate and personalized recommendations by balancing the user's current desires with their overall preferences (An et al., 2019).

Chapter 3

Literature Review

3.1 Introduction to Literature Review

The literature review in this thesis provides a thorough examination of the research landscape surrounding news recommender systems (NRS). The primary objective is to synthesize existing knowledge, identify gaps in the current research, and connect it with the theoretical foundations that support this study. This chapter is organized into several sections, each focusing on a distinct aspect of the literature relevant to this research. The chapter identifies key research gaps that this research aims to address. This chapter sets the stage for the methodology and experimental sections that follow. The insights from this review will be instrumental in developing new user representation model and assessing how well it can enhance the quality of news recommendations.

3.2 News Recommender Systems and its Components

In this section, we discuss in detail the NRS architecture and how popular NRS models implement the components in their respective architectures.

3.2.1 News Encoder

The **news encoder** is responsible for generating vector representations of news articles. Transformer-based models like BERT are commonly employed to encode the textual content of articles into dense vectors, capturing their semantic meaning. Deep learning-based approaches automatically learn representations from the original input, often leveraging Natural Language Processing (NLP) models. For example, CNN-based methods like NPA Wu et al. (2019b) use CNNs to generate contextual word representations and attention mechanisms to select important words. Self-attention-based methods like NRMS Wu et al. (2019c) and MINS Wang et al. (2022) employ multi-head self-attention to learn word representations and form news representations. Multi-view methods like NAML Wu et al. (2019a) treat news components separately with attentive mechanisms to combine representations. LSTUR An et al. (2019) uses CNNs and attention to encode titles and incorporates topic information. CAUM Qi et al. (2022) utilizes self-attention to model news individually and aggregate representations.

3.2.2 User Encoder

The **user encoder** generates embeddings that represent a user's preferences based on their reading history. Deep learning-based methods focus on automatically identifying useful patterns from user behaviours. Attention-based methods like NPA and NAML Wu et al.

(2019b;a) use news attention to select and weight informative news articles based on relevance to model user preferences. Self-attention methods like NRMS Wu et al. (2019c) use self-attention to capture interactions between browsed news and attention to select informative news for learning user representations. CNN-based methods like CAUM's Candi-CNN integrate candidate news into local behaviour modelling with CNNs to capture patterns between adjacent clicks. Sequence modelling methods like LSTUR, CAUM's Candi-SelfAtt, and MINS use RNN/GRU/multi-channel GRU to model sequential dependencies in behaviours and capture user interests. Representation aggregation methods like LSTUR, CAUM, and MINS An et al. (2019); Qi et al. (2022); Wang et al. (2022)integrate candidate news representations, combining short-term and long-term features while aggregating diverse data sources through techniques such as concatenation and additive attention.

3.2.3 Click Prediction

The **click predictor** estimates the likelihood of a user clicking on a given news article. It uses the embeddings generated by the news and user encoders to calculate a relevance score, typically through a similarity function like the dot product. Models like NPA, NRMS, NAML, LSTUR, CAUM, and MINS Wu et al. (2019b;c;a); An et al. (2019); Qi et al. (2022); Wang et al. (2022)share a commonality in utilizing similar scoring functions, typically involving the dot product between user and news representations. Variations in loss functions used during training enhance accuracy, with some models incorporating negative sampling techniques for improved training efficiency. Models like CAUM employ specific loss functions like BPR loss, while others like MINS use mini-batch gradient descent on log-likelihood loss. Unique methodologies such as LSTUR's pseudo-K + 1-

way classification problem and CAUM's contextual training with random sampling are also employed to handle specific scenarios.

3.3 Biases in News Recommender Systems (NRS)

While ideal news recommender systems (NRSs) aim to provide high-quality, personalized recommendations, research highlights various biases that can affect their effectiveness. Bias in NRS refers to systematic preferences that arise either from the training data or the system's design, leading to unfair or irrelevant recommendations (Chen et al., 2023).

One common issue is popularity bias, where a small fraction of popular items dominate user interactions. Models trained on this skewed data tend to over-recommend popular items, exacerbating their exposure while under-representing less popular ones (Steck, 2011). Similarly, exposure bias occurs when certain articles are more likely to be encountered due to demographics, platform dynamics, or prior recommendations, limiting the representation of less exposed items in the model (Joachims and Radlinski, 2007).Sentiment bias arises when NRSs favor articles with specific emotional tones, often recommending content with negative sentiment, which can reinforce user beliefs (Alam et al., 2022). Stance bias occurs when systems favor articles that take a specific stance, reflecting the viewpoint distribution in the training data and potentially limiting exposure to diverse perspectives (Alam et al., 2022).Score bias is seen when certain articles consistently receive higher recommendation scores, regardless of their relevance to user preferences. Explainability methods have shown that these high scores are not always based on strong content alignment (Möller and Padó, 2024). Geometric bias relates to the structure of user and article embeddings, where limited diversity in user representations leads to certain

articles receiving higher scores simply because they are closer to the average user in the embedding space (Möller and Padó, 2024).

3.4 Disentanglement Techniques

Disentanglement techniques are essential for ensuring that distinct user interests are represented clearly without overlap, improving the personalization and relevance of recommendations. This section reviews key methods for disentanglement in user representations.

One common technique is to minimize overlap between interest vectors by penalizing their dot products. The dot product measures the similarity between two vectors, and Ma et al. (2019) proposed using orthogonal regularization to penalize dot products between latent factors, promoting distinct representations by maintaining independence between interest vectors. Cosine similarity, which measures the angle between vectors, is also used to ensure orthogonality in user interest vectors, helping represent distinct dimensions of user behavior. Mikolov (2013) demonstrated the effectiveness of this approach in the word2vec model, capturing semantic relationships between words and extending it to user preferences.Minimizing mutual information, which quantifies the information shared between variables, is another approach to disentanglement. Chen et al. (2016) applied this method in InfoGAN to learn distinct, non-overlapping latent variables, ensuring independent representation of different factors in the data.

By applying these techniques, models can better capture the complexity of user interests, leading to more personalized and accurate recommendations in news recommender systems.

3.5 Research Gaps

Despite the advancements in NRS, several research gaps remain. Traditional single-vector user representations fail to capture the diverse and evolving interests of users, resulting in biased and less relevant recommendations. Existing deep learning models, while powerful, often do not adequately address the need for multi-interest representations that can adapt to changing user preferences. ZHANG et al. (2023) highlight the importance of accurately capturing user preferences and modeling news and users in news recommendation systems. They propose a new framework that uses topic and knowledge embedding to address these challenges, emphasizing the need for more nuanced user representations. Ma et al. (2023) introduce an unsupervised pre-training paradigm for effective user behavior modeling. They emphasize the necessity of improved user representations to predict user preferences accurately, thereby addressing the evolving interests of users.

Existing models do not adequately address the need for multi-interest representations. There is a need for models that can dynamically capture multiple user interests and adapt to their evolving preferences. Wang et al. (2023) discuss the limitations of current deep learning models in capturing diverse user interests. They propose using dynamic representations and contrastive user modelling to address these challenges and improve recommendation accuracy. Issues like popularity, exposure, sentiment, and stance biases remain significant challenges. These biases can distort the recommendation process, leading to unfair and potentially harmful outcomes for users. Alam et al. (2022) highlight how these biases can reinforce existing user beliefs and limit exposure to diverse perspectives. Möller and Padó (2024) how these biases can lead to unfair recommendation scores and reduced diversity in user representation.

Chapter 4

Methodology

Introduction

This chapter elaborates on the proposed multi-interest model and also states the assumptions and limitations.

4.1 Model Architecture

The architecture of the Multi-Interest Model developed in this work is centred around three core components: **the News Encoder, the User Encoder, and the Click Prediction module**. These components work together to capture user interests, represent news articles, and predict the likelihood of a user clicking on a particular news article. Below is a detailed explanation of each component and how they integrate to form the complete model.

4.1.1 News Encoder

The News Encoder converts raw news articles into dense vector representations for processing within the recommendation system. This process begins by using pre-trained transformer models, such as BERT, to extract rich semantic features from the text. These features are critical for capturing nuanced information necessary for accurate recommendations. The encoding process involves several steps.

Initially, the model processes sequential news embeddings and attention masks, which help focus on the most relevant content. The input embeddings are shaped as (B, N, S, D), where B is the batch size, N is the number of articles, S is the sequence length, and D is the embedding dimensionality. The attention mechanism emphasizes key information within these sequences. A pooling operation then aggregates this information across the sequence dimension, producing a condensed vector per article. These pooled vectors may be further processed through a fully connected neural network (the "head") to adjust their dimensionality, ensuring they meet the required output features.

The final output consists of news vectors reshaped to $(B, N, \textit{out\_features})$, encapsulating the semantic essence of the articles for comparison with user interest vectors. A collapsed attention mask is also generated, reducing its dimensionality to $(B, N, 1)$ for further processing. These outputs are crucial for matching news content with user preferences, enhancing the recommendation system's performance.

4.1.2 User Encoder

The User Encoder captures the diverse range of user interests through a multi-interest representation approach. Unlike traditional models that represent interests with a single

vector, this encoder produces multiple vectors, each reflecting a different aspect of the user's preferences. This method better accounts for the complexity and variability in user behavior. Each user's interests are represented by a matrix of dimensions $D \times n$, where D is the embedding dimensionality and n is the number of distinct interests. This matrix format enables a richer representation of user preferences compared to single-vector models.

To generate these interest vectors, the model employs additive attention mechanisms applied to the user's historical interactions with news articles. The attention mechanism uses a two-layer neural network: the first layer projects the input into a hidden space with a non-linear activation (e.g., *tanh*), and the second layer generates normalized attention scores, producing weighted summaries of the input embeddings. These summaries are then processed through a fully connected neural network (the "head") to adjust their shape and scale, making them suitable for comparison with news vectors. In our Multi-interest model, we have one pooler and one head corresponding to every interest making it a 'n' pooler, 'n' heads architecture. The final output is a set of interest vectors shaped $(B, \textit{num\_interests}, D)$, where B is the batch size, *num_interests* is the number of interest vectors, and D is the dimensionality. The encoder may also return attention weights, highlighting the aspects of user behavior that were most influential.This approach effectively captures the complex nature of user interests, enabling more personalized and accurate recommendations.

4.1.3 Click Prediction

The Click Prediction module estimates the likelihood of a user clicking on a news article. Traditional methods use a single user and news vector, computing their dot product to

assess similarity. However, in our multi-interest model, the user is represented by a matrix $\mathbf{U} \in \mathbb{R}^{n \times D}$, where n is the number of interests and D is the dimensionality. The candidate news article is represented by a vector $\mathbf{c} \in \mathbb{R}^{D \times 1}$. The click prediction score s' is the maximum score across all interest vectors:

$$s' = \max(\mathbf{U} \cdot \mathbf{c})$$

This approach ensures the model selects the most relevant user interest for accurate prediction. The module implements this by comparing user interest vectors with the candidate news vector, selecting the highest score. The `DotScoring` class handles this, taking user matrix $\mathbf{U}$ and news vector $\mathbf{c}$, reshaping them for batch processing, and computing the dot product. If enabled, both vectors are normalized before scoring. The final tensor, $(B, N, 1)$, reflects similarity, with the highest score used for the click prediction. This method effectively manages the complexity of multi-interest user representation, enhancing prediction accuracy.

In summary, the model integrates the News Encoder, User Encoder, and Click Prediction modules. The News Encoder converts articles into vectors, the User Encoder generates multiple interest vectors, and the Click Prediction module computes the dot product between user and news vectors, selecting the maximum score to predict clicks. This integration ensures accurate user interaction predictions.

4.1.4 Scoring Loss and Disentanglement Loss Functions

In multi-interest user representation models, two key loss functions—scoring loss and disentanglement loss—optimize both recommendation accuracy and the diversity of user

representations.

Scoring Loss: The **Binary Cross Entropy (BCE)** loss measures the error between predicted probabilities (user interaction likelihood) and actual outcomes (e.g., clicks). This loss helps rank items based on engagement probability. During training, logits (raw outputs) are used directly with BCE for numerical stability. Scoring loss adjusts model weights to improve user interaction predictions, evaluated through metrics like AUC and NDCG.

Disentanglement Loss: The **disentanglement loss** ensures distinct user interest vectors by computing cosine similarity between pairs of interest vectors and penalizing high similarity. By minimizing off-diagonal elements in the similarity matrix, this loss encourages diverse, non-overlapping interest vectors, capturing different aspects of user behavior.

Relationship Between Scoring and Disentanglement Losses: The total loss combines both **scoring** and **disentanglement** losses, balanced by hyperparameters λ_s (scoring loss weight) and λ_u (disentanglement loss weight):

$$\text{Total Loss} = \lambda_s \cdot \text{Scoring Loss} + \lambda_u \cdot \text{Disentanglement Loss}$$

where $\lambda_s + \lambda_u = 1$. Tuning these values balances recommendation accuracy with interest diversity. L2 regularization is applied to prevent overfitting, and gradient descent is used for optimization. This balance allows the model to accurately rank relevant items while maintaining distinct user interest vectors, enhancing personalized recommendations.

4.2 Assumptions and Limitations

The experimental design is based on several key assumptions. It assumes that user behaviour is consistent over time and that the datasets used are representative of real-world interactions. Additionally, it is presumed that the selected evaluation metrics accurately reflect the model's performance, fairness, and potential biases.

However, there are limitations to this approach. The quality and representativeness of the data could impact the results, particularly if the data is biased or lacks diversity. Scalability may also be a concern when applying these methods to larger datasets. Moreover, the model might face challenges related to the cold start problem, where recommendations for new users or items are less accurate. Lastly, while these experiments focus on a news recommender system, the findings may not generalize to other domains or types of recommendation systems.

Chapter 5

Experiments, Results and Discussion

5.1 Introduction

This chapter presents a comprehensive overview of the experiments conducted to evaluate the effectiveness of multi-interest user representations within a news recommender system. The experiments are designed to address three key objectives, each targeting different aspects of the recommendation process. These studies aim to compare traditional single-vector user representations with the proposed multi-interest approach while analyzing their effects on various performance metrics. We begin with introducing the data which will be used for these experiments. We then discuss the hardware and software specifications and then the experimental design. The sections after that will discuss each experiment and it's corresponding implementation details, results and discussion in detail.

5.2 Data Preparation and Processing

5.2.1 Overview of MIND Dataset

The primary data source used in this research is the **Microsoft News Dataset (MIND)**. It is specifically designed for studies in news recommendation systems. This large-scale dataset is developed from the user behaviour logs of one million users. These users were randomly selected based on whether they recorded at least five news clicks over six weeks. All user data is anonymized into anonymized IDs to protect privacy. This is done by using secure hashing. The dataset provides rich information on user interactions with news articles, primarily in terms of click logs and impression logs. Each news article is associated with metadata, including titles, abstracts, bodies, unique identifiers, and manually tagged categories and subcategories. This dataset is further divided into three sets: the training set which includes 2,186,683 samples, the validation set which contains 365,200 samples, and the test set which comprises 2,341,619 samples. Wu et al. (2020)

5.2.2 Data pre-processing

We will begin by discussing how the MIND dataset was processed for use in the experiments. Several preprocessing steps were performed on the MIND dataset to prepare it for training and evaluating the news recommendation model.

1. **Data Loading:** The dataset was loaded from tab-separated files (`news.tsv`, `behaviors.tsv`) into pandas DataFrames, structuring the news articles and user behaviors for easy access during preprocessing.

2. **Data Cleaning and Handling Missing Data:** Duplicate records in user interactions and news articles were removed to avoid biases. Cold-start users (those without interaction history) were filtered out to ensure sufficient data for training. Missing values in critical fields like article titles or user click logs were addressed through imputation or exclusion, with missing text data filled with an empty string (") before tokenization.

3. **Category Indexing:** News categories and subcategories were numerically indexed, converting categorical labels into numerical indices for efficient model processing.

4. **Text Normalization and Tokenization:** Text was normalized (lowercase, stop words removed, punctuation standardized) and then tokenized using a pre-trained transformer model (e.g., uclanlp/newsbert). Sequences were padded to a fixed length (`SEQ_LEN=50`) to ensure uniformity for batch processing in neural networks. The tokenized data was stored in new DataFrame columns for easy model input.

5. **Embedding Precomputation:** Precomputed embeddings for news titles and abstracts were generated using a transformer model and stored in the DataFrame, capturing semantic information for use in model training.

6. **Dataset Splitting:** The dataset was split into training (2,186,683 samples), validation (365,200 samples), and test sets (2,341,619 samples) based on predefined configurations. This ensured effective training, fine-tuning, and fair evaluation of the model.

7. **Negative Sampling:** Negative samples (non-clicked articles) were selected for each positive user interaction (click), aiding the model in learning to distinguish relevant from irrelevant news articles.

5.3 Implementation Details: Software, Libraries, Tools, and Hardware Specifications

Software Libraries and Data Management: The multi-interest model is implemented using Python 3.8+ and built on PyTorch, which supports dynamic computation graphs and GPU acceleration. Key libraries include `NumPy` for numerical operations, `Pandas` for data manipulation, and `Matplotlib` for visualizations. `Scikit-learn` is used for dimensionality reduction, and `TQDM` provides progress bars for monitoring. Configuration is managed using `Omegaconf`, `DotMap`, and `YAML` files, while `Weights and Biases (WandB)` is used for real-time tracking of experiments. Data is stored in `pickle` and `CSV` formats, containing pre-encoded news articles and user interaction histories. The model processes historical data with a minimum history length of 1, history length of 25, and sequence length of 50.

Model and Training Configuration: User preferences are captured through multiple interest vectors (n interest vectors per user). Embedding dimensions are set at 256 for both titles and users, with a backbone dimension of 768. The model employs a dropout rate of 0.1 and weight decay of 1×10^{-4} for regularization. The scoring function uses the dot product for similarity, while cosine similarity manages disentanglement to ensure distinct user interest vectors. Training is configured with a batch size of 64, a learning rate of 0.0001, and four negative samples per positive instance. Data shuffling and two workers optimize data loading, with checkpoints saved after each epoch.

Operating System, Hardware, and Local Machine Usage: The server runs on Fedora 40 (Kernel 6.9.9) and features dual Intel Xeon E5-2650 v4 CPUs (48 logical processors), 251 GiB of RAM, and four NVIDIA GeForce GTX 1080 Ti GPUs (44 GB total

GPU memory). CUDA 12.5 and NVIDIA driver 555.58.02 ensure compatibility with deep learning tasks. All computationally intensive processes are handled on the server, while the local machine is used to connect, manage workflows, and monitor training progress.

5.4 Experimental Setup

This thesis employs a series of experiments designed to evaluate the effectiveness of multi-interest user representations within a news recommender system. The experiments are structured around four key objectives:

Click Prediction Performance: The first experiment focuses on comparing traditional single-vector user representations with multi-interest representations. This comparison will be evaluated using metrics such as AUC, NDCG, and MRR to determine if multi-interest representations enhance click prediction accuracy.

Disentanglement of User Interests: The second experiment addresses the disentanglement of user interests. By integrating a disentanglement loss function, this experiment aims to achieve a clear separation between different user interests. The effectiveness of this approach will be evaluated both visually, through embedding visualization, and quantitatively, using metrics like cosine similarity.

Impact on Recommendation Fairness and Diversity: In the third experiment, the impact of multi-interest representations on recommendation fairness and diversity will be examined. This study will assess how these representations affect the distribution of recommendation scores, focusing on reducing bias and increasing diversity. The analysis will compare mean scores and variance for single-vector versus multi-interest representations.

5.5 Experiment 01: Click Prediction Performance

5.5.1 Objective

The first experiment focuses on evaluating the impact of multi-interest user representations on the accuracy of click predictions within a news recommender system. Click prediction is a critical component of recommender systems, as it directly influences the relevance and effectiveness of the recommendations presented to users. The goal is to determine whether adopting multi-interest user representations can improve click prediction accuracy compared to traditional single-vector representations.

5.5.2 Experimental Design

We will train a series of click prediction models, starting with a traditional single-vector model, which is a special case of the multi-interest model with $N = 1$. The multi-interest model generates multiple embeddings for each user, representing different aspects of their interests. We will experiment with N values of 5, 10, 25, and 50 to assess the impact of varying interest vectors. All models will be trained on the same dataset for fair comparison, and performance will be evaluated using metrics such as AUC, NDCG, and MRR to determine the effectiveness of the multi-interest approach.

5.5.3 Implementation and Evaluation Details

The implementation of this experiment involves several key steps:

Implementation Details: The single-vector model uses one vector to represent user preferences, while the multi-interest model employs multiple vectors to capture different

aspects of user interests. Both models share the same configuration: title embedding dimension ($title\_emb\_dim$) of 256, user embedding dimension ($user\_emb\_dim$) of 256, backbone dimension ($d\_backbone$) of 768, dropout rate ($p\_dropout$) of 0.1, and 6 training epochs (n_epochs) with a learning rate of 0.0001. The multi-interest model varies the number of interest vectors ($num\_interests$) with values of 5, 10, 25, and 50. The single-vector representation model serves as the baseline, providing a benchmark against which the multi-interest model's performance is measured.

Evaluation Metrics: The models' performance is primarily evaluated using key metrics: Area Under the ROC Curve (`AUC`), which assesses the model's ability to distinguish between classes; Normalized Discounted Cumulative Gain (`NDCG@10` and `NDCG@5`), which evaluates ranking quality by comparing predicted rankings with the ideal ranking for the top 10 and 5 positions; and Mean Reciprocal Rank (`MRR`), reflecting the average of the reciprocal ranks of the first relevant item. These metrics are central to assessing the model's effectiveness in ranking and classification tasks. While other metrics, such as Recall (`rec`), Precision (`prec`), Test Loss, Train Loss, Accuracy (`acc`), and Click-Through Rate at 1 (`CTR@1`) are also logged, they are not detailed here as they serve as supplementary indicators. For all metrics except loss, higher values indicate better performance.

5.5.4 Results

In this section, we present and compare the results for both the Single Vector Model and the Multi-Interest Model.

A) Single Vector Model

Description: The Single Vector Model, a special case of the multi-interest model with $N = 1$, aggregates all user preferences into a single embedding vector to predict user

clicks on news articles. It was trained for 6 epochs with a learning rate of 0.0001, using Binary Cross Entropy (BCE) as the loss function. No disentanglement loss was applied in this model.

Results:

Metric	Value
rec	0.042
ndcg@10	0.430
ndcg@5	0.366
prec	0.047
test_loss	0.363
acc	0.898
train_loss	0.218
auc	0.686
ctr@1	0.213
mrr	0.384

Table 5.1: Performance Metrics for Single Vector Model (N=1)

Note: The metrics in the table represent the highest values observed across all epochs during training.

Observation: The Single Vector Model produces an AUC of 0.686, NDCG@10 of 0.430, and MRR of 0.384. Secondary metrics include recall (0.042), precision (0.047), CTR@1 (0.213), and an accuracy of 0.898. The train loss is 0.218, and the test loss is 0.363.

B) Multi-Interest Model

Description: The Multi-Interest Model represents a user's preferences with multiple vectors, each capturing a different facet of their interests. It was trained for 6 epochs with a learning rate of 0.0001, using Binary Cross Entropy (BCE) as the loss function. No disentanglement loss was applied. The model was tested with various N values, including

5, 10, 25, and 50.

Results:

Metric	N = 1	N = 5	N = 10	N = 25	N = 50
rec	0.042	0.128	0.156	0.152	0.162
ndcg@10	0.430	0.431	0.437	0.442	0.438
ndcg@5	0.366	0.367	0.371	0.377	0.374
prec	0.047	0.106	0.121	0.122	0.124
test_loss	0.363	0.362	0.364	0.361	0.367
acc	0.898	0.883	0.884	0.885	0.883
train_loss	0.218	0.443	0.443	0.443	0.444
auc	0.686	0.691	0.695	0.701	0.696
ctr@1	0.213	0.216	0.218	0.221	0.220
mrr	0.384	0.387	0.390	0.395	0.395

Table 5.2: Performance Metrics for Multi-Interest Model (Varying N)

Note: The metrics in the table represent the highest values observed across all epochs during training.

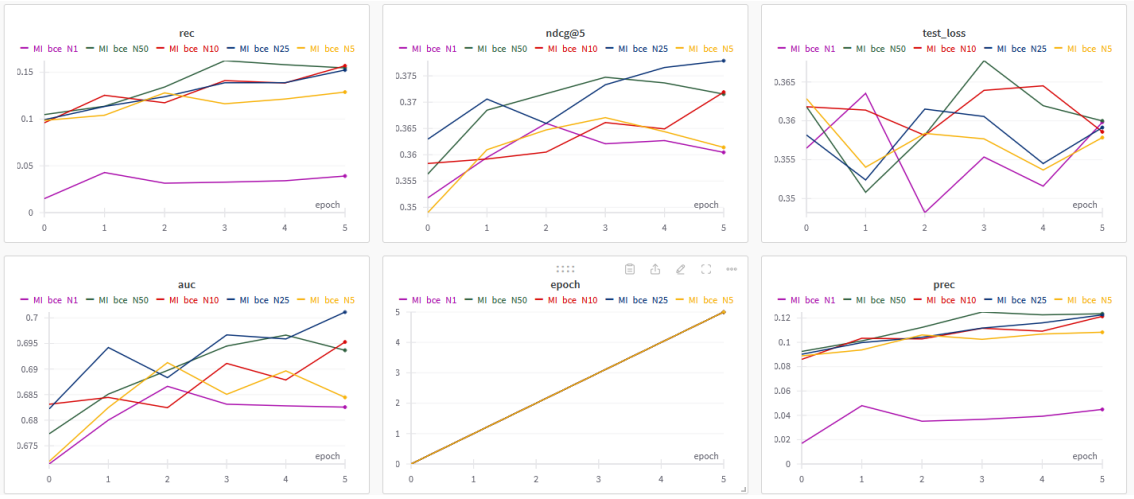

Figure 5.1: Performance metrics for Multi-Interest Model across different values of N.

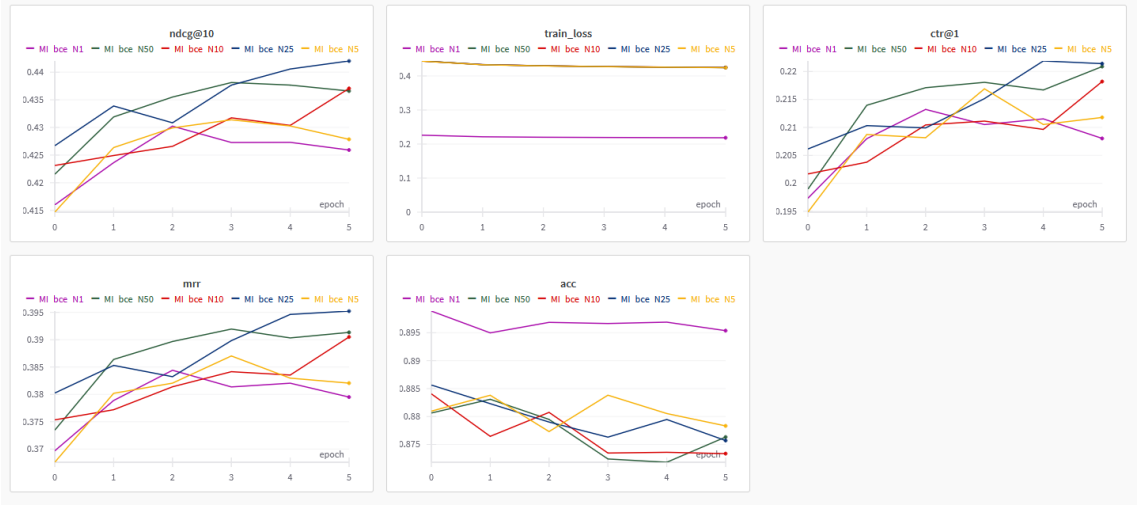

Figure 5.2: Performance metrics for Multi-Interest Model across different values of N.

Observation: The Multi-Interest Model improves performance compared to the Single Vector Model as the number of interest vectors (`N`) increases. For `N=5`, recall improves to 0.128 and precision to 0.106. At `N=10`, AUC increases to 0.695, and NDCG@10 reaches 0.437. The best performance is observed at `N=25`, where AUC reaches 0.701, NDCG@10 is 0.442, and MRR improves to 0.395. CTR@1 increases to 0.221 at `N=25`. At `N=50`, the model shows minimal further improvement, with AUC remaining at 0.696 and NDCG@10 slightly decreasing to 0.438, suggesting that performance plateaus at higher `N` values.

The graphs illustrate these trends, showing a steady increase in key metrics like recall, NDCG, and AUC up to `N=25`, followed by a plateau at `N=50`. Notably, the standard deviation of metrics like train and test loss remains stable across epochs, indicating minimal overfitting as `N` increases.

5.5.5 Discussion

A) Single-Vector Model: The Single Vector Model shows moderate performance in ranking tasks, with an AUC of 0.686, indicating a reasonable ability to distinguish between clicked and non-clicked items. However, the NDCG@10 of 0.430 and MRR of 0.384 highlight limitations in placing relevant items at top positions. Low recall (0.042) and precision (0.047) suggest the model struggles to identify all relevant items and retrieves some irrelevant ones. The CTR@1 of 0.213 shows that the most relevant item is placed first in about 21% of cases, which is modest. The accuracy of 0.898 likely reflects class imbalance rather than true ranking performance. The small difference between train loss (0.218) and test loss (0.363) indicates stability but also suggests that the single-vector approach is too simplistic to capture more complex user preferences, especially in scenarios requiring higher recall.

B) Multi-Interest Model: The Multi-Interest Model, which uses multiple vectors to represent user interests, shows clear improvements, particularly at `N=25`. The AUC of 0.701 demonstrates better discrimination between clicked and non-clicked items, while NDCG@10 (0.442) and MRR (0.395) reflect enhanced ranking performance. The model's ability to retrieve more relevant items is evident in the recall increase from 0.042 to 0.152 at `N=25`, along with improved precision (0.122), leading to better recommendations. The CTR@1 of 0.221 shows a modest gain, but the impact diminishes as `N` increases further. By `N=50`, performance plateaus, with AUC slightly decreasing to 0.696 and NDCG@10 to 0.438. This suggests that adding more vectors introduces complexity without further gains, potentially due to overfitting. Despite this, the stable train (0.443) and test (0.361) losses indicate that the model does not overfit, though increasing complexity beyond `N=25` provides minimal additional benefit.

Conclusion: The experiment demonstrates that the Multi-Interest Model outperforms the Single Vector Model, particularly at N=25, by capturing the user's diverse preferences more accurately, leading to improvements in key metrics such as AUC, NDCG, and MRR. However, the diminishing returns at higher N values suggest that while multi-interest representations are superior, optimal performance requires careful tuning of the number of interest vectors to balance complexity and accuracy.

5.6 Experiment 02: Disentanglement of User Interests

5.6.1 Objective

This experiment evaluates the effectiveness of disentangling user interests in the multi-interest user representation model. The goal is to minimize overlap between different interest vectors, ensuring clear and distinct representations of user interests. We introduce a cosine similarity-based disentanglement loss during model training and analyze its impact on the resulting user embeddings.

5.6.2 Experimental Design

This experiment focuses on disentangling user interests by applying a disentanglement loss during training. The loss function reduces overlap between interest vectors, encouraging distinct representation of user preferences.

Key Steps: We conduct a quantitative analysis and visualization to assess the impact of disentanglement. For the **quantitative analysis**, we compare the performance of the

multi-interest model with and without the disentanglement loss across different N values (number of interest vectors) using metrics such as recall, NDCG@10, NDCG@5, precision, test loss, accuracy, train loss, AUC, CTR@1, and MRR. For **visualization**, we assess the separation of user interests by plotting histograms of user embeddings, examining the distribution of cosine similarities between each user's representation and the mean representation across all users. Additionally, we compute average pairwise similarities within each user's interest vectors to evaluate their distinctness and use t-SNE plots of user representations for further insight.

5.6.3 Implementation and Evaluation Details

Quantitative Analysis

Implementation Details: In this experiment, a cosine similarity-based disentanglement loss is introduced to encourage the separation of user interest vectors. We have explained it in detail in the Methodology section. Initially, various λ_s values are tested with $N = 10$, and $\lambda_s = 0.9$ is found to be optimal. The model is then evaluated across different N values with λ_s fixed at 0.9.

Evaluation Details: We evaluate the model using key metrics such as recall, NDCG@10, NDCG@5, precision, test loss, accuracy, train loss, AUC, CTR@1, and MRR, providing a comprehensive assessment of click prediction performance and the effectiveness of disentangling user interests.

Visualization

Implementation Details: This part focuses on visualizing the separation of user interests achieved by the multi-interest model, with and without the disentanglement loss, using cosine similarity histograms, average pairwise cosine similarity histograms, and t-SNE plots.

Cosine Similarity Histograms: These histograms display the distribution of cosine similarities between all individual interest vectors and the mean interest vector computed from all users' interest vectors. By examining these distributions, we can analyze how similar or dissimilar the users' interest vectors are to the mean user interest. **Average Pairwise Cosine Similarity Histograms:** These histograms show the distinctness of interest vectors within each user. Pairwise cosine similarities between all interest vectors for each user are averaged and plotted to evaluate how distinct these vectors are. **t-SNE Plots:** t-SNE is used to visualize high-dimensional interest vectors in 2D space. The process involves flattening the interest vectors, calculating t-SNE with different parameters, and plotting scatter plots. The best distinction between user interests was achieved with perplexity = 60 and learning rate = 500, and these results are included in the results section.

Evaluation Details: The visualizations are evaluated to determine how effectively the disentanglement loss enhances the separation of user interests. Comparing histograms and t-SNE plots with and without the disentanglement loss reveals the degree of separation achieved.

5.6.4 Results

A) Quantitative Analysis

In this section, we present our results for Quantitative Analysis.

Metric	$\lambda_s = 0.99$	$\lambda_s = 0.95$	$\lambda_s = 0.9$	$\lambda_s = 0.5$	$\lambda_s = 0.1$
rec	0.058	0.052	0.057	0.049	0.028
ndcg@10	0.426	0.430	0.428	0.428	0.400
ndcg@5	0.361	0.366	0.364	0.363	0.336
prec	0.063	0.056	0.060	0.052	0.031
test_loss	0.361	0.363	0.364	0.363	0.371
acc	0.897	0.898	0.897	0.898	0.899
train_loss	0.431	0.414	0.392	0.228	0.046
auc	0.685	0.686	0.687	0.686	0.653
ctr@1	0.209	0.213	0.209	0.209	0.185
mrr	0.380	0.385	0.380	0.381	0.355

Table 5.3: Performance metrics for N=10 across different values of λ_s.

Observation: The value of $\lambda_s = 0.9$ was selected for subsequent analysis as it provided the best overall performance, with stable metrics such as precision, recall, and AUC without significant loss in test accuracy or increase in loss.

Metric	**N=5**	**N=10**	**N=25**	**N=50**
rec	0.050	0.057	0.047	0.062
ndcg@10	0.430	0.428	0.434	0.438
ndcg@5	0.366	0.364	0.369	0.374
prec	0.056	0.060	0.051	0.064
test_loss	0.360	0.364	0.362	0.365
acc	0.897	0.897	0.897	0.897
train_loss	0.408	0.407	0.415	0.442
auc	0.687	0.688	0.692	0.693
ctr@1	0.217	0.209	0.216	0.224
mrr	0.386	0.380	0.386	0.394

Table 5.4: Performance metrics for different values of N with $\lambda_s = 0.9$.

Observation: Performance with $\lambda_s = 0.9$ for Different N

With $\lambda_s = 0.9$ fixed, the performance metrics generally improve as the number of interest vectors N increases. Recall shows its highest value at N=50 (0.062), while AUC also improves progressively, reaching 0.693 at N=50. NDCG@10 increases slightly from 0.430 at N=5 to 0.438 at N=50, with similar improvements seen in MRR, which peaks at 0.394 at N=50. Secondary metrics such as precision (0.064) and CTR@1 (0.224) are highest at N=50. Overall, the metrics show a trend of improvement with increasing N.

Observation: Comparison of Values With and Without Disentanglement Loss for Different N

Comparing the results from Table 5.2 (without disentanglement loss) and Table 5.4 (with disentanglement loss and $\lambda_s = 0.9$), we observe that without disentanglement loss, metrics like AUC (0.701 at N=25) and NDCG@10 (0.442 at N=25) show higher values. However, with disentanglement loss, AUC reaches 0.693 at N=50, which is slightly lower than without disentanglement. Metrics such as recall and precision also see slight decreases with disentanglement loss, while NDCG@10 and AUC remain relatively stable as N increases.

B) Visualizations

I) Cosine Similarities (All Users)

Description: In this section, we analyze the distribution of cosine similarities between all individual interest vectors and the mean representation across all users. The x-axis represents the cosine similarity, ranging from 0 to 1. The y-axis represents the frequency of these similarities, with scales that vary based on the number of users. The log scale

plots provide additional insight into the distribution, particularly at the extremes.

Results:

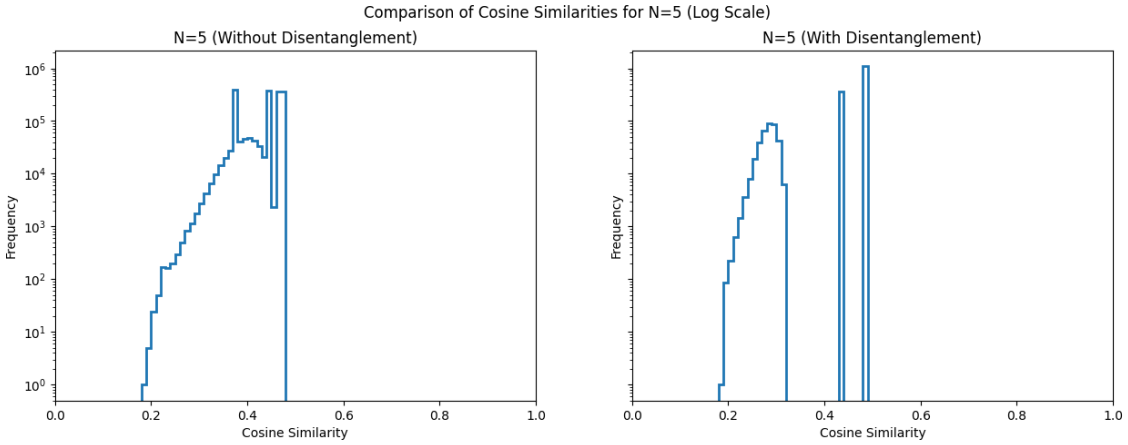

Figure 5.3: Comparison of Cosine Similarities for $N = 5$ on a Log Scale (Left: Without Disentanglement, Right: With Disentanglement).

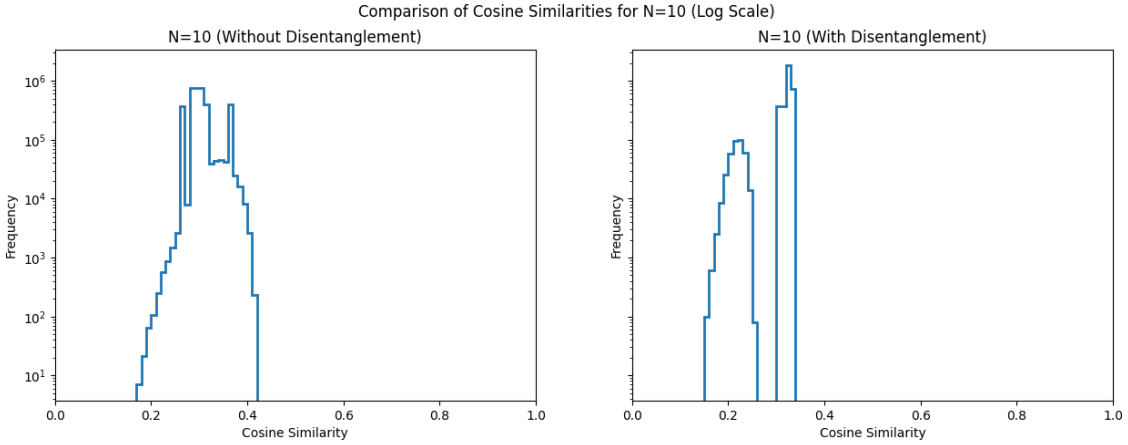

Figure 5.4: Comparison of Cosine Similarities for $N = 10$ on a Log Scale (Left: Without Disentanglement, Right: With Disentanglement).

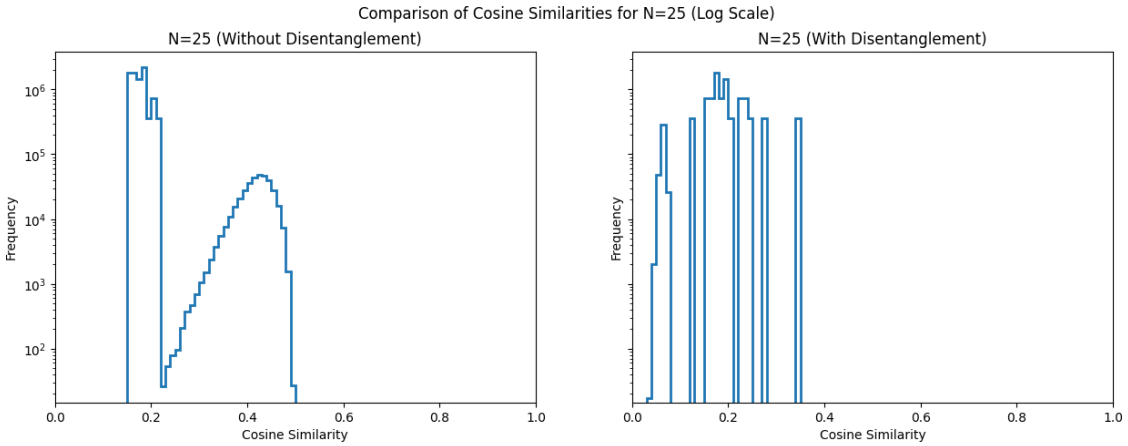

Figure 5.5: Comparison of Cosine Similarities for $N = 25$ on a Log Scale (Left: Without Disentanglement, Right: With Disentanglement).

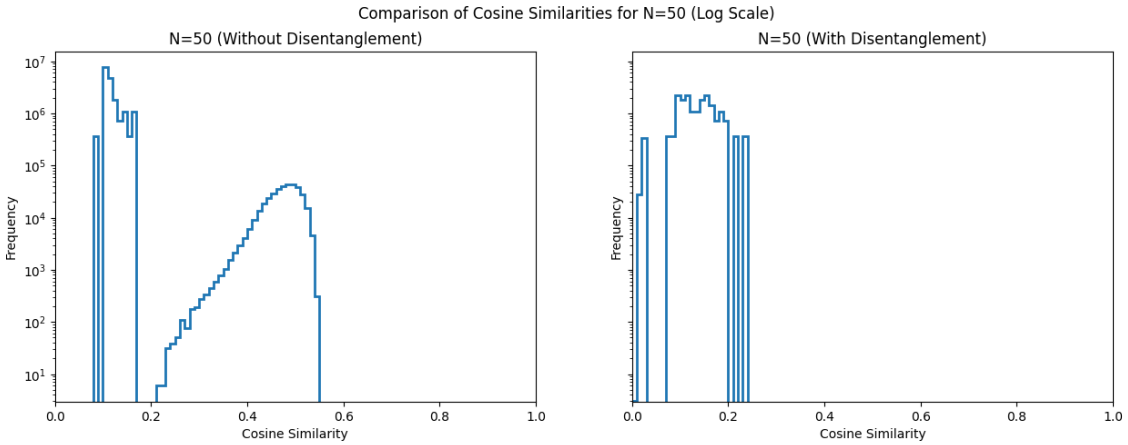

Figure 5.6: Comparison of Cosine Similarities for $N = 50$ on a Log Scale(Left: Without Disentanglement, Right: With Disentanglement).

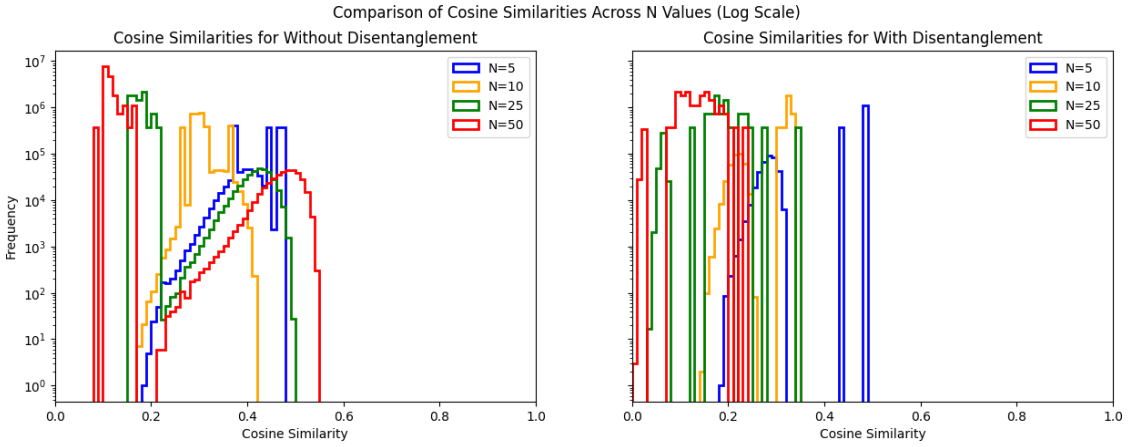

Figure 5.7: Combined Comparison of Cosine Similarities Across Different N Values on Log Scale(Left: Without Disentanglement, Right: With Disentanglement).

Observation:

For $N = 5$, the histogram without disentanglement shows a broad spectrum around mid-similarity scores. With disentanglement, the spectrum becomes narrower and more defined and additional distinct peaks are observed. At $N = 10$, the histogram without disentanglement shifts slightly to the left compared to $N = 5$, however, the broad spectrum is still present. With disentanglement, the spectrum narrows and peaks are observed. For $N = 25$, without disentanglement, apart from the broad spectrum, we notice additional distinct peaks. With disentanglement, the broad spectrum almost vanishes and now only distinct peaks are observed. The distribution also significantly shifts to the left. At $N = 50$, the histogram without disentanglement shows a broad spectrum as well as distinct peaks, while with disentanglement, the peaks sharpen and shift towards the left.

II) Average Pairwise Cosine Similarities

Description: This section visualizes the average pairwise cosine similarities within each user's representation vectors. For each user, the cosine similarities between all pairs of interest vectors are calculated and then averaged to obtain a single value. The x-axis shows cosine similarity values, ranging from 0 (completely disentangled) to 1 (identical vectors), while the y-axis indicates the frequency of these values across users. This visualization evaluates the distinctness of each user's interest representations.

Results: We present the visualizations obtained in this section.

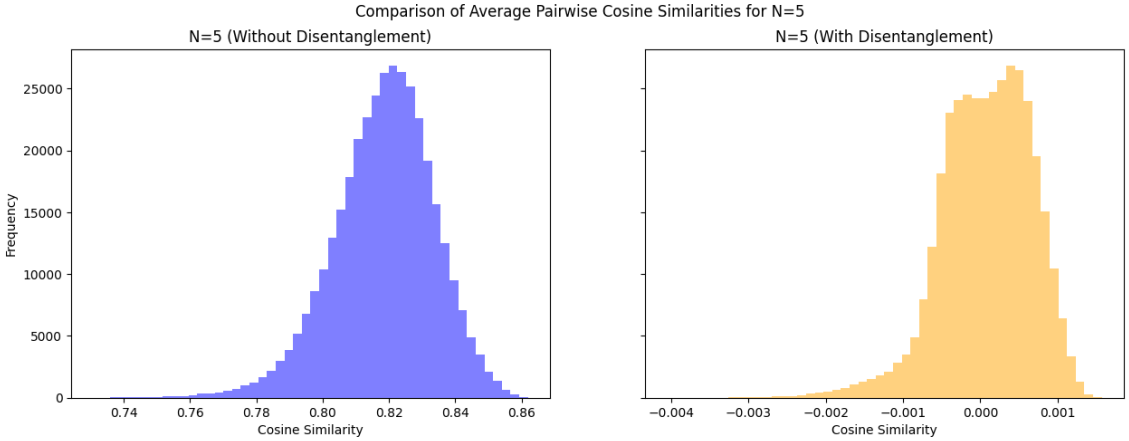

Figure 5.8: Average Pairwise Cosine Similarities for $N = 5$(Left: Without Disentanglement, Right: With Disentanglement).

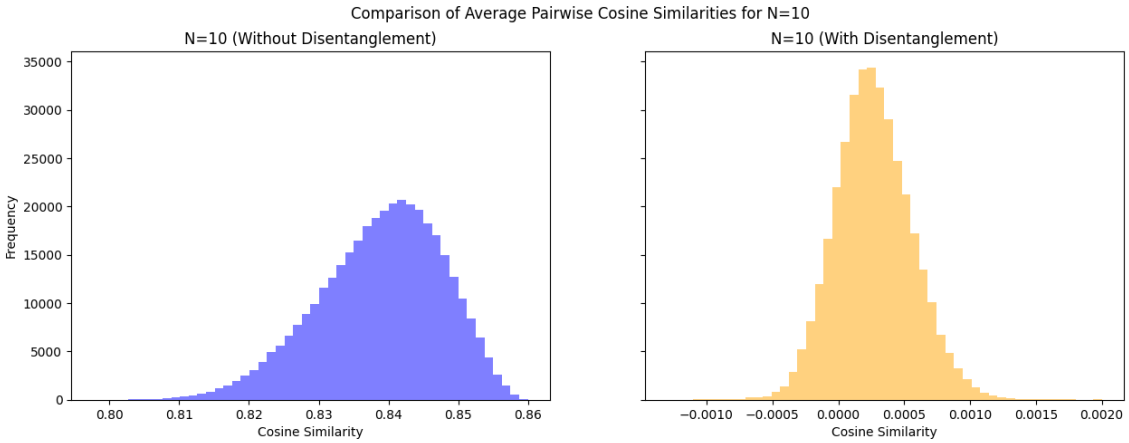

Figure 5.9: Average Pairwise Cosine Similarities for $N = 10$(Left: Without Disentanglement, Right: With Disentanglement).

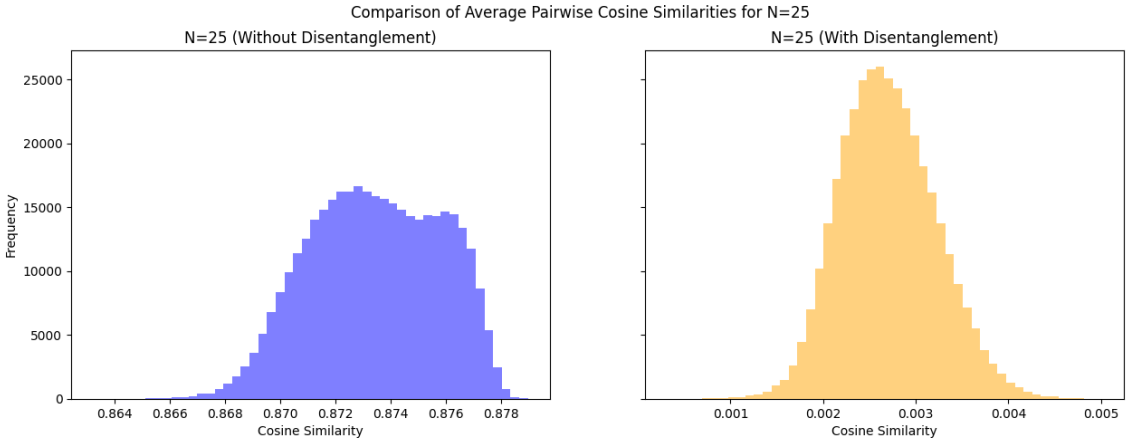

Figure 5.10: Average Pairwise Cosine Similarities for $N = 25$(Left: Without Disentanglement, Right: With Disentanglement).

.

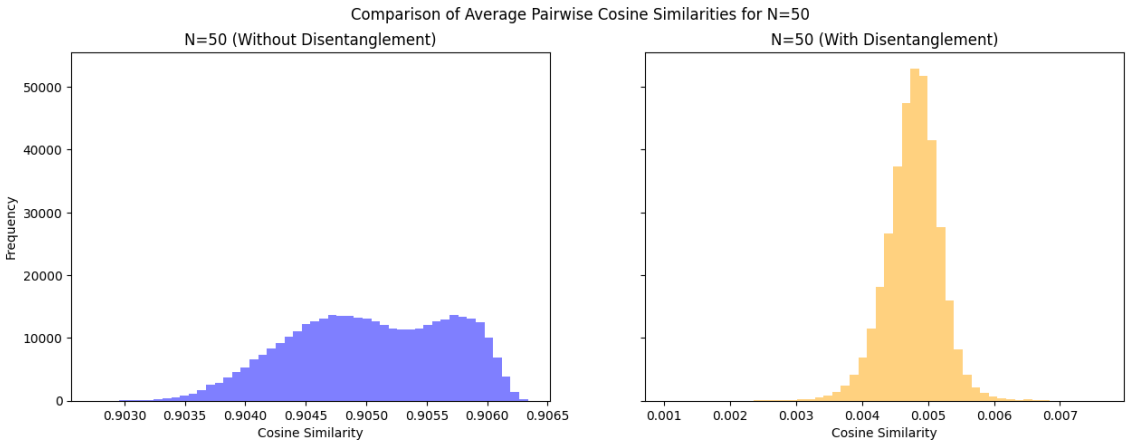

Figure 5.11: Average Pairwise Cosine Similarities for $N = 50$(Left: Without Disentanglement, Right: With Disentanglement).

Observations: For $N = 5$, the histogram of average pairwise cosine similarities without disentanglement shows a wider spread and lies in the range of 0.86 to 0.87. With disentanglement, the histogram tightens, displaying a more concentrated peak centring around the range 0.001 to 0.004. At $N = 10$, the histogram without disentanglement is more scattered, while with disentanglement, the peak sharpens, indicating a narrower distribution of similarity scores. For $N = 25$, the histogram without disentanglement shows a broader distribution, but with disentanglement, the distribution becomes much tighter with a higher concentration around a central peak. At $N = 50$, the histogram without disentanglement has a broad spread, while with disentanglement, the histogram narrows into a sharp peak.

III) t-SNE Plots

Description: In the plots, the x-axis and y-axis represent the two principal components resulting from this dimensionality reduction. We experiment with several parameter combinations for t-SNE, and the final plots are based on the best-performing parameters:

perplexity = 60 and learning rate = 500. Each color-coded cluster corresponds to different user interest vectors, providing insight into how distinct these vectors are when using different numbers of interest vectors N and the disentanglement loss ($\lambda_s = 0.9$).

Visualizations:

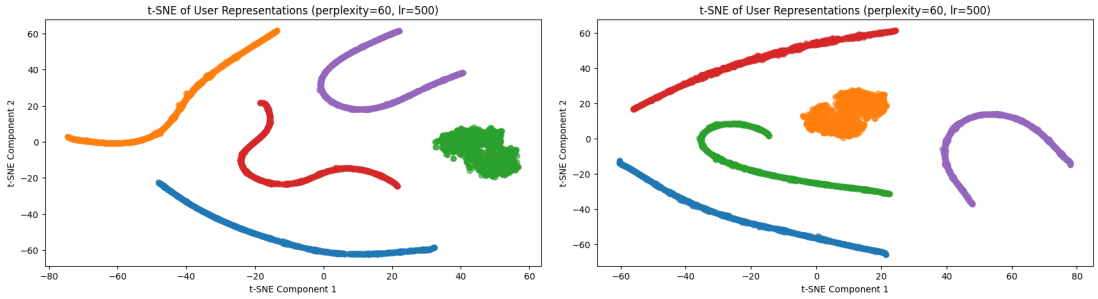

Figure 5.12: t-SNE Plot for User Interest Representations with $N = 5$ for perplexity (60,500) (Left: Without Disentanglement, Right: With Disentanglement).

.

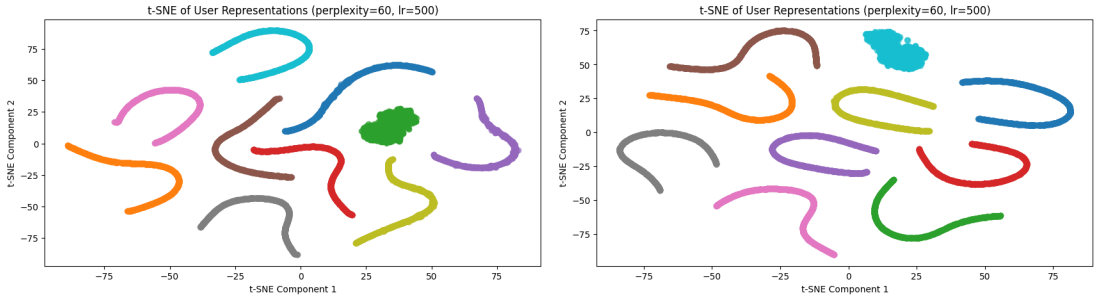

Figure 5.13: t-SNE Plot for User Interest Representations with $N = 10$for perplexity (60,500) (Left: Without Disentanglement, Right: With Disentanglement).

.

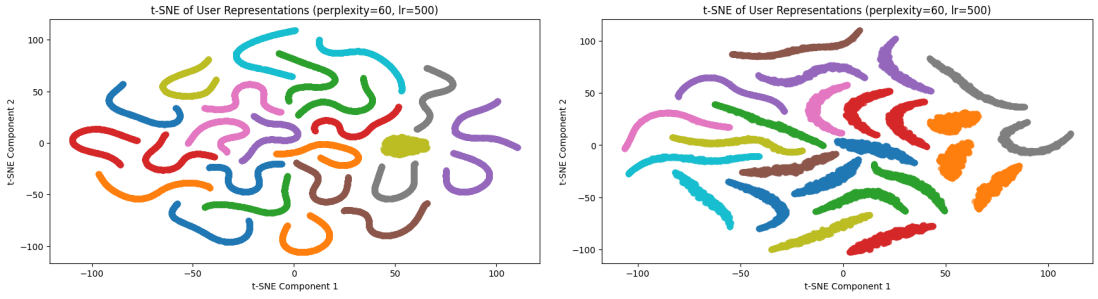

Figure 5.14: t-SNE Plot for User Interest Representations with $N = 25$for perplexity (60,500) (Left: Without Disentanglement, Right: With Disentanglement).

.

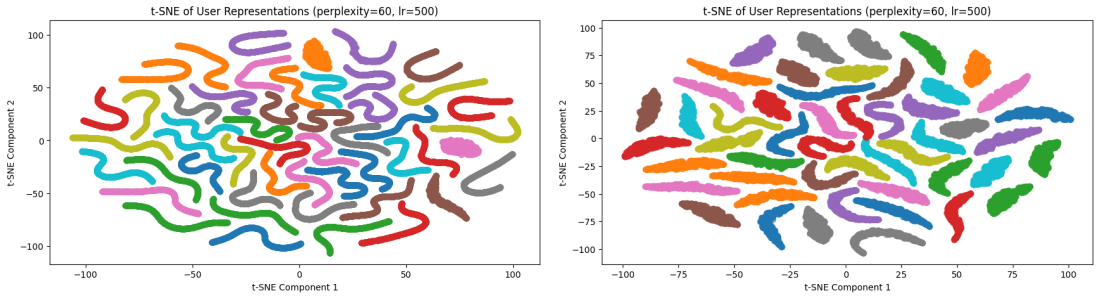

Figure 5.15: t-SNE Plot for User Interest Representations with $N = 50$for perplexity (60,500) (Left: Without Disentanglement, Right: With Disentanglement).

Observations: The t-SNE plots illustrate that as the number of interest vectors N increases, user interest representations become more distinct and better separated in the two-dimensional space. This effect is particularly evident when the disentanglement loss is applied ($\lambda_s = 0.9$). The clustering observed in the right-hand plots (with disentanglement) for each N suggests that the interest vectors are more effectively separated, enhancing the model's ability to distinguish between different user preferences.

5.6.5 Discussion

Quantitative Analysis

The incorporation of disentanglement loss ($\lambda_s = 0.9$) produces mixed results across key performance metrics like **AUC**, **NDCG@10**, and **precision**. While disentanglement improves the separation of user interest vectors, its effect on recommendation accuracy varies depending on the number of interest vectors (`N`).

AUC remains strong but peaks at 0.701 without disentanglement for `N=25`, compared to 0.693 with disentanglement for `N=50`. This slight decrease suggests that disentanglement may slightly reduce the model's ability to rank relevant items above non-relevant

ones. **NDCG@10** follows a similar trend, peaking at 0.442 without disentanglement (`N=25`), but stabilizing slightly lower at 0.438 with disentanglement for `N=50`.**Precision** and **recall** also show the highest values at `N=50`, but both slightly decrease with disentanglement, indicating that the model may lose some focus on the most relevant items, introducing noise. However, secondary metrics like **CTR@1** and **MRR** consistently improve with increasing `N`, though disentanglement slightly reduces their peak performance.

Implications: While disentanglement enhances interest separation, it may come at the cost of precision and top-ranked recommendation performance, as shown by slight decreases in **AUC**, **NDCG@10**, and **precision**. This suggests that disentanglement, while useful for promoting diversity, may spread attention too broadly across content, affecting the model's accuracy in highlighting the most relevant items. Striking a balance between diversity and precision through careful tuning of λ_s and λ_u could improve both performance and fairness in recommendations.

Visualizations

The visualizations provide deeper insight into how disentanglement affects user interest representations within the multi-interest model. By analyzing cosine similarities, average pairwise similarities, and t-SNE plots, we gain a clearer understanding of how disentanglement influences the distinctness and diversity of user interest vectors.

I) Cosine Similarities

The cosine similarity histograms reveal how the interest vectors are aligned relative to the mean user representation. Without disentanglement, we observe a broad spectrum of similarity scores, indicating that the interest vectors lie in a wide range of directions relative to the mean vector. This broad spread suggests that interest vectors are scattered

across a large space, with no clear alignment or clustering around specific directions. As a result, the model struggles to form distinct, well-separated representations of user interests. With disentanglement, the distribution becomes narrower and distinct peaks emerge. These peaks indicate that the interest vectors are now more aligned along specific angles relative to the mean vector. This suggests that the disentanglement loss has enforced a structure where the interest vectors fall closer to a straight line or specific angular directions, creating well-defined clusters of user preferences. By concentrating the vectors around certain angles, disentanglement reduces the range of possible vector directions, leading to more distinct, concentrated interest vectors. The shift in the distribution to the left with disentanglement also implies that the vectors are becoming less similar to the mean representation, indicating stronger separation between user interests and the general profile. This reduction in cosine similarity reflects a greater degree of specialization in how the model represents different interests.

II) Average Pairwise Cosine Similarities

The average pairwise cosine similarity histograms further illustrate how disentanglement affects the diversity of interest vectors within individual users. Without disentanglement, the cosine similarity values between a user's interest vectors are relatively high, ranging between 0.6 and 0.9. This wide range suggests that the different interest vectors are relatively similar to each other, indicating significant overlap or redundancy in the user's representation. With disentanglement, the average pairwise cosine similarities shift dramatically, falling within a much narrower range—typically between -0.002 and 0.004. This extremely low similarity range suggests that the disentanglement process has successfully made the interest vectors almost completely distinct from one another. The much smaller range of pairwise cosine similarities confirms that disentanglement achieves

its goal of minimizing redundancy among interest vectors.

III) t-SNE Plots

The t-SNE plots offer a visual representation of how user interest vectors are organized in two-dimensional space. Without disentanglement, the clusters representing different user interest vectors are scattered and unorderly, indicating significant overlap between interest groups. The colors in these plots correspond to different interest vectors, and their dispersion shows that the model struggles to distinguish between user preferences, leading to a blending of interest groups. With disentanglement, the t-SNE plots reveal much tighter clustering. The interest groups, represented by different colors, are more closely packed together, forming distinct clusters that are better separated from one another. This clustering behavior indicates that disentanglement has successfully separated the different interest vectors, organizing them into well-defined groups that correspond to distinct user preferences. This enhanced separation helps the model focus more precisely on different interest groups, improving its ability to recommend relevant content for each specific interest. However, it's important to balance the strength of disentanglement, as overly sharp clustering could risk over-segmenting interests, potentially leading to less flexible recommendations when user preferences overlap or evolve over time.

Conclusion:

This experiment evaluated the impact of disentanglement loss on the multi-interest user representation model in a news recommender system. The results show that disentanglement effectively improves the separation of interest vectors. Quantitative metrics, particularly with $\lambda_s = 0.9$, indicated stable or enhanced performance in predicting user clicks. Visualizations confirmed these findings, showing clearer interest representations. While disentanglement is not a complete solution, it signifi-

cantly improves the clarity and effectiveness of user interest modelling.

5.7 Expertiment 03: Impact on Recommendation Fairness and Diversity

5.7.1 Objective

This experiment evaluates the impact of multi-interest user representations on recommendation fairness and diversity by analyzing the distribution of recommendation scores. The aim is to determine whether multi-interest representations lead to reduced bias (indicated by flatter slopes in the mean score distribution) and increased diversity (indicated by wider score variance) compared to single-vector representations.

5.7.2 Experimental Design

To evaluate the impact of multi-interest user representations on recommendation fairness and diversity, we adopt an approach inspired by Möller and Padó at IMS. This experiment consists of two main analyses:

1. **Mean Recommender Score Analysis:**
   We compute the mean recommendation scores across all users and news articles for both single-vector and multi-interest user representations. This analysis focuses on comparing the slope of the mean recommendation scores. A flatter slope would suggest that no particular news article receives consistently higher scores, indicating a

reduction in bias. In essence, the flatter the slope, the more balanced the recommendation scores, implying that the model is not overly favoring any specific news content.

2. **Variance (Standard Deviation) Analysis:**
   We assess the variance (standard deviation) of the recommendation scores to evaluate recommendation diversity. A wider distribution of scores would indicate that the recommender system is offering a more varied set of recommendations, catering to a broader range of user preferences. In theory, as bias is reduced, the diversity of recommendations should increase, leading to a more balanced recommendation output with a wider spread around the mean.

Both analyses are conducted for single-vector and multi-interest representations. We aim to observe whether multi-interest representations lead to flatter mean score slopes (indicating reduced bias) and wider standard deviation (indicating increased diversity) compared to single-vector representations.

5.7.3 Implementation and Evaluation Details

Implementation

In this experiment, pre-saved user and candidate embeddings are first loaded into the system. To ensure efficient processing while maintaining a representative sample of data, a random selection of 10,000 user embeddings and 3,500 candidate embeddings is made. Once the embeddings are loaded, the next step involves adjusting the embedding dimensions for compatibility. Since the multi-interest representation model may have a different

number of interest vectors per user, the candidate embeddings are repeated or truncated accordingly to ensure proper alignment with the user embeddings.

After adjusting the embeddings, recommendation scores are calculated using cosine similarity for each user-candidate pair. These scores are then standardized with a sigmoid function, ensuring that they fall within the range of 0 to 1, which simplifies interpretation and analysis. The final step involves preparing the data for analysis. The mean and variance (standard deviation) of the recommendation scores are computed to assess two key aspects: bias, which is evaluated through the slope of the mean scores, and diversity, which is assessed through the width of the score distribution.

Evaluation

The evaluation process compares single-vector and multi-interest representations, both with and without disentanglement loss, focusing on bias and diversity. First, the mean score slopes are analyzed to evaluate bias. A flatter slope in the mean score distribution indicates reduced bias, and comparisons across different configurations help to determine the effect of multi-interest representations, particularly when disentanglement is applied, on bias reduction. Additionally, variance (standard deviation) analysis is conducted to assess recommendation diversity. A wider distribution of scores suggests a more diverse set of recommendations, whereas a decrease in variance could imply that the model is focusing too heavily on specific interests, potentially reducing diversity. Visualization is key in evaluating the impact of multi-interest representations. Side-by-side comparisons of mean score slopes and standard deviation are used to examine how these representations influence bias reduction and diversity. This approach ensures a thorough evaluation of both quantitative and qualitative aspects of recommendation fairness and diversity.

5.7.4 Results

Figure 5.16: Recommendation Scores for $N = 1$.

.

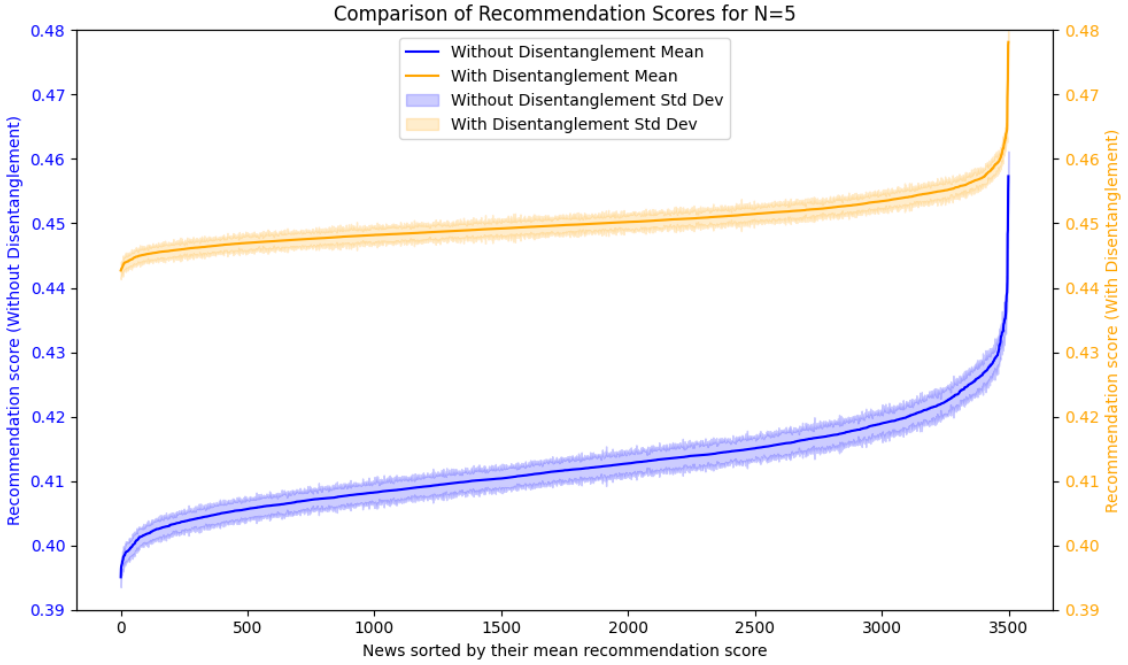

Figure 5.17: Comparison of Recommendation Scores for $N = 5$ (Without Disentanglement & With Disentanglement).

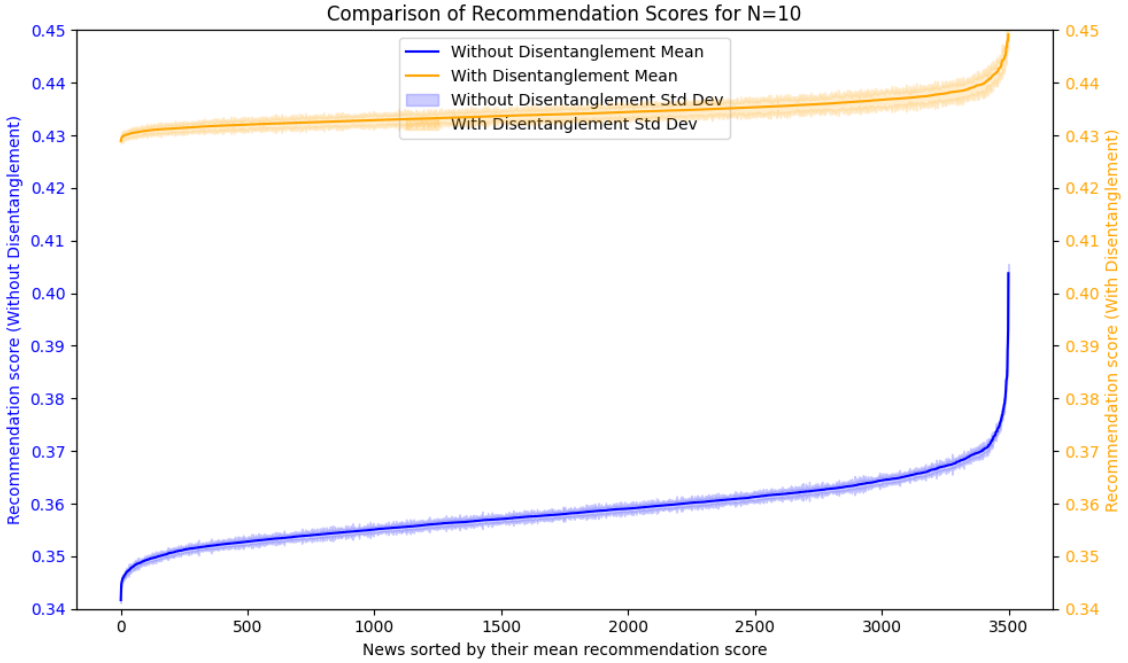

Figure 5.18: Comparison of Recommendation Scores for $N = 10$ (Without Disentanglement & With Disentanglement).

.

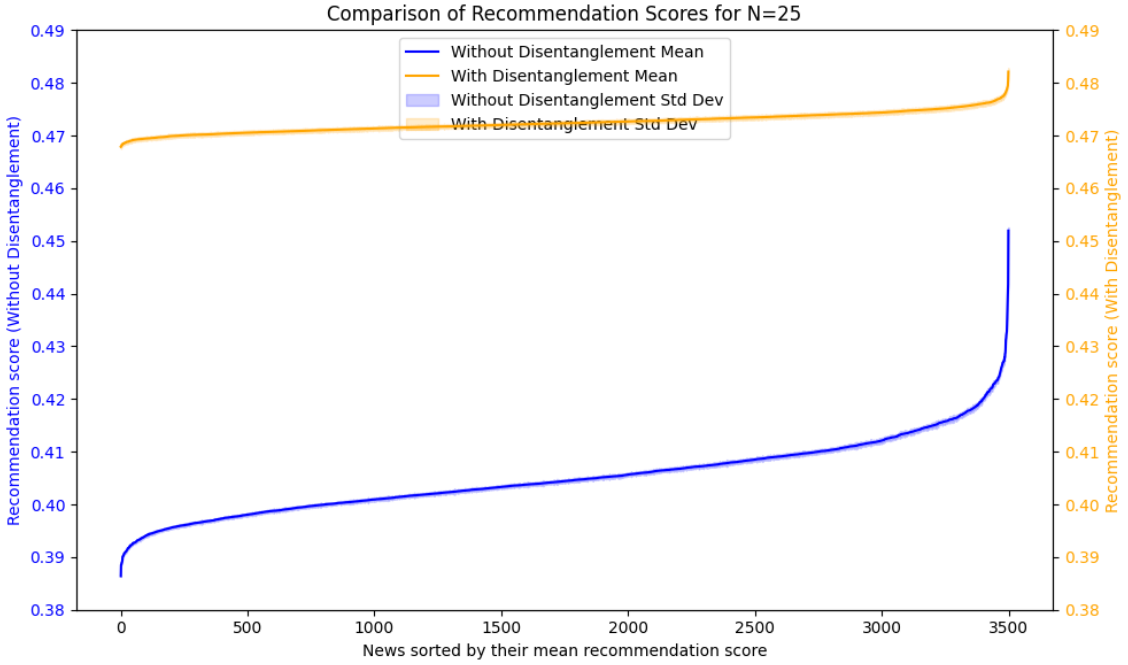

Figure 5.19: Comparison of Recommendation Scores for $N = 25$ (Without Disentanglement & With Disentanglement).

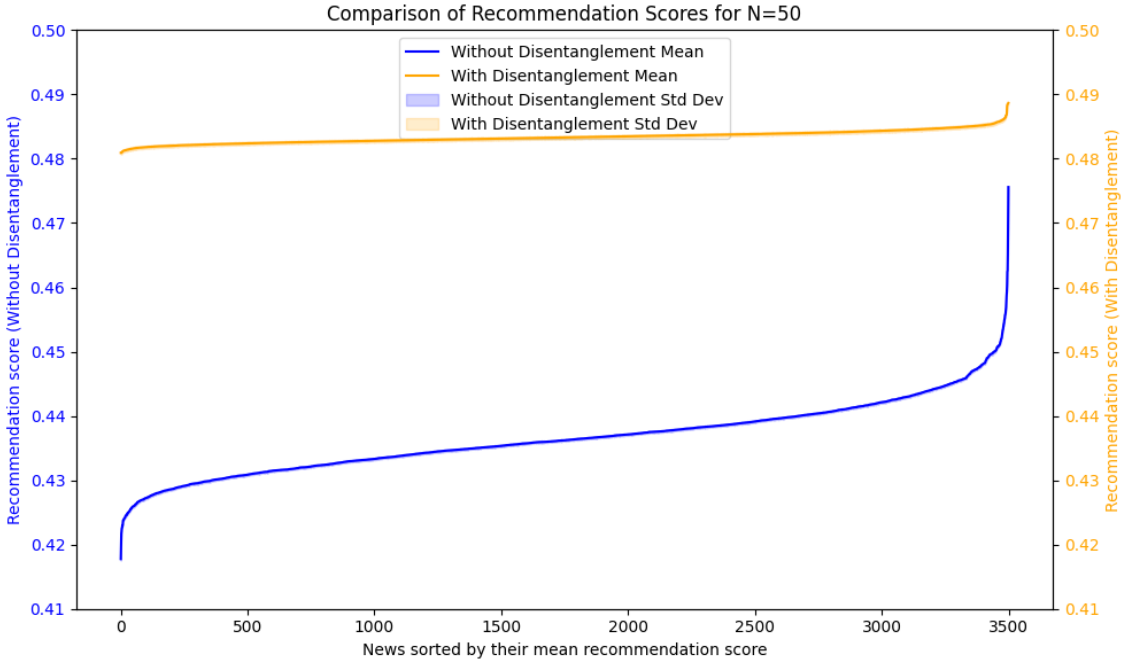

Figure 5.20: Comparison of Recommendation Scores for $N = 50$ (Without Disentanglement & With Disentanglement).

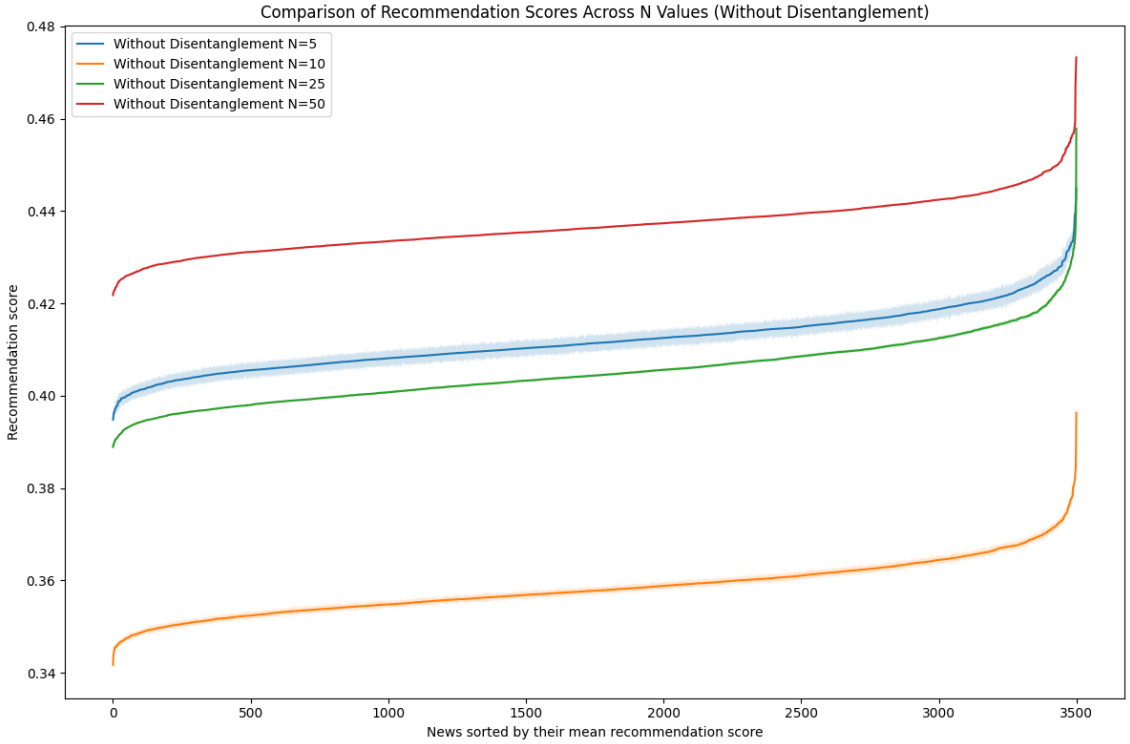

Figure 5.21: Comparison of Recommendation Scores Across N Values (Without Disentanglement).

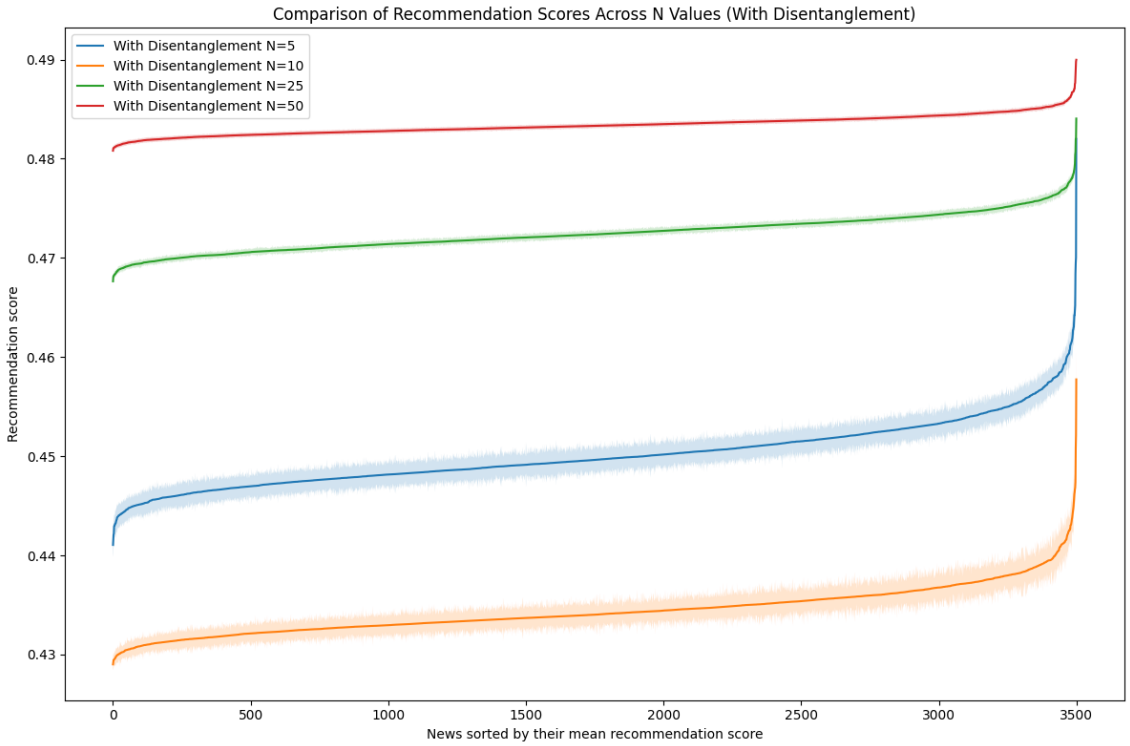

Figure 5.22: Comparison of Recommendation Scores Across N Values (With Disentanglement).

5.7.5 Observation

For **N=5**, without disentanglement, we observe that the mean recommendation score ranges from approximately 0.39 to 0.42, with a steep slope indicating significant differences in how news articles are ranked. With disentanglement, the score range becomes more constrained, from 0.42 to 0.45, and the slope flattens, suggesting a more uniform ranking of news articles. For **N=10**, without disentanglement, the score range shifts slightly upwards, from 0.36 to 0.44, with a somewhat flatter slope compared to N=5, indicating less variability in recommendation scores. With disentanglement, the range narrows further, from 0.43 to 0.46, with an even flatter slope, reflecting more consistent recommendations across articles.

At **N=25**, without disentanglement, the score range becomes 0.38 to 0.46, with a more noticeable flattening of the slope, suggesting that as N increases, the differences in how news articles are ranked become less pronounced. With disentanglement, the range is tighter, from 0.44 to 0.47, with a significantly flatter slope, indicating that the model is ranking articles more uniformly, potentially reducing bias. Finally, for **N=50**, without disentanglement, the mean score range increases to 0.42 to 0.47, but the slope remains relatively flat, showing minimal variation in ranking compared to lower N values. With disentanglement, the score range narrows further, from 0.46 to 0.48, and the slope is nearly flat, suggesting that the model is treating most articles similarly, with minimal bias towards any particular content.

5.7.6 Discussion

Mean Recommendation Scores:

The graphs show distinct trends for both with and without disentanglement as **N** increases. Without disentanglement, as the number of interest vectors (**N**) grows, the range of recommendation scores broadens initially (from **N=5** to **N=10**), then stabilizes around **N=25** and **N=50**. At **N=5**, the model shows a steeper slope, indicating more bias toward specific news articles. As **N** increases, the slope flattens, suggesting more balanced score distribution and reduced bias. With disentanglement, the slope flattens significantly across all **N** values. At **N=50**, the slope is nearly flat, indicating more equal treatment of news articles and further bias reduction compared to non-disentangled versions.

The flatter slope, especially in the disentangled versions, indicates a more balanced recommendation system, with less bias toward specific content. A wider score range in the non-disentangled versions suggests more extreme ranking differences, while a narrower range with disentanglement implies more uniform treatment of news items, reducing over-recommendation of specific content.

Variance and Diversity:

Despite increasing **N** to enhance diversity, we observe a consistent reduction in the standard deviation (variance) of recommendation scores, particularly with disentanglement. This suggests the model becomes more uniform in its recommendations. Ideally, higher **N** should lead to greater diversity by capturing more user interests, but the reduced variance suggests the model is becoming overly uniform, possibly at the cost of diversity.

This reduction in variance could reflect better alignment between user preferences and recommendations, meaning the model is consistently recommending relevant items. However, it could also indicate an oversimplification of user preferences, treating too many items as equally relevant. Further analysis is needed to understand whether this reduction in variance enhances or detracts from the recommendation experience.

Conclusion:

Increasing N improves the balance in recommendation scores by flattening the slope, reducing bias toward specific content. Disentanglement further amplifies this effect, leading to more uniform, less biased recommendations. However, the reduction in variance as N increases, especially with disentanglement, raises concerns about reduced diversity in recommendations. While the model becomes more consistent in assigning scores, this uniformity may limit content variety. Further research is needed to assess whether this trend enhances user satisfaction or results in oversimplified recommendations that fail to capture the full range of user preferences.

Chapter 6

Conclusion and Future Works

6.1 Conclusion

This thesis aimed to explore the impact of multi-interest user representations and disentanglement loss on improving recommendation fairness and diversity in a news recommender system. Through a series of experiments, we analyzed how increasing the number of user interest vectors and incorporating disentanglement loss affected various performance metrics, user engagement, and recommendation quality. Three experiments were conducted to measure click prediction performance, disentanglement of user interests, and the trade-offs between recommendation fairness and diversity.

Experiment 1 compared the Single-Vector Model and the Multi-Interest Model. The results showed that the Multi-Interest Model outperformed the Single-Vector Model across key metrics like AUC, NDCG@10, and MRR, especially at `N=25`. While the Multi-Interest Model demonstrated improved ability to capture diverse user preferences, performance plateaued beyond `N=25`, indicating that excessive model complexity did not

translate into further improvements.

Experiment 2 investigated the effects of applying disentanglement loss ($\lambda_s = 0.9$) to the Multi-Interest Model. Although disentanglement improved the distinctness of interest vectors, its impact on recommendation accuracy varied depending on the number of interest vectors. A slight decrease in key metrics such as AUC and NDCG@10 was observed with disentanglement. Visualizations like cosine similarities and t-SNE plots showed clearer separation of user preferences but hinted at potential trade-offs between diversity and precision.

Experiment 3 focused on the mean recommendation score and variance. As `N` increased, recommendation scores became more balanced, flattening the slope of the mean scores and indicating reduced bias toward specific content. However, despite expectations, variance decreased, particularly with disentanglement, raising concerns about diminished diversity. While the model became more consistent, it may have oversimplified user preferences, potentially limiting the variety of content recommendations.

6.2 Future Work

Although this thesis demonstrated the benefits of multi-interest representations and disentanglement loss, several open areas for future research remain:

1. **Balancing Diversity and Precision:** The experiments revealed that disentanglement reduced bias but also decreased diversity in recommendations. Future work should focus on optimizing the trade-off between precision and diversity by experimenting with hybrid models or alternative regularization techniques that maintain both fairness and variety in recommendations.

2. **User Satisfaction and Long-Term Effects:** While this thesis measured performance metrics, understanding the long-term effects on user satisfaction and engagement is crucial. Future research should explore how multi-interest models and disentanglement affect user experiences over time, particularly in contexts where diverse and personalized recommendations are essential.

3. **Dynamic Interest Modeling:** User preferences are dynamic and evolve over time. Integrating temporal elements into the multi-interest model could improve its adaptability, allowing it to respond better to changing user behaviors and interests.

4. **Cross-Dataset Evaluation:** To ensure the generalizability of the findings, future work should evaluate the models on multiple datasets with varying characteristics, such as different types of content or user demographics. This will provide a more comprehensive understanding of the model's applicability across different domains.

6.3 Final Remarks

In conclusion, this thesis explored the efficacy of multi-interest representations and disentanglement in enhancing news recommender systems. While the results showed that increasing the number of user interest vectors and incorporating disentanglement reduced bias and improved fairness, they also indicated potential challenges in maintaining recommendation diversity. Striking the right balance between personalization, fairness, and diversity remains a crucial challenge for future research. By refining these techniques and incorporating user feedback, recommender systems can continue evolving toward more equitable and engaging experiences for all users.

Statement of Authorship

Erklärung (Statement of Authorship)

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst habe und dabei keine andere als die angegebene Literatur verwendet habe. Alle Zitate und sinngemäßen Entlehnungen sind als solche unter genauer Angabe der Quelle gekennzeichnet. Die eingereichte Arbeit ist weder vollständig noch in wesentlichen Teilen Gegenstand eines anderen Prüfungsverfahrens gewesen. Sie ist weder vollständig noch in Teilen bereits veröffentlicht. Die beigefügte elektronische Version stimmt mit dem Druckexemplar überein.

[1]

(Aditi Godbole)

[1]Non-binding translation for convenience: This thesis is the result of my own independent work, and any material from work of others which is used either verbatim or indirectly in the text is credited to the author including details about the exact source in the text. This work has not been part of any other previous examination, neither completely nor in parts. It has neither completely nor partially been published before. The submitted electronic version is identical to this print version.

Bibliography

Mehwish Alam, Andreea Iana, Alexander Grote, Katharina Ludwig, Philipp Müller, and Heiko Paulheim. Towards analyzing the bias of news recommender systems using sentiment and stance detection. In *Companion Proceedings of the Web Conference 2022*, pages 448–457, 2022.

Mingxiao An, Fangzhao Wu, Chuhan Wu, Kun Zhang, Zheng Liu, and Xing Xie. Neural news recommendation with long-and short-term user representations. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 336–345, 2019.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Joeran Beel, Bela Gipp, Stefan Langer, and Corinna Breitinger. Paper recommender systems: a literature survey. *International Journal on Digital Libraries*, 17:305–338, 2016.

Joan Borràs, Antonio Moreno, and Aida Valls. Intelligent tourism recommender systems: A survey. *Expert systems with applications*, 41(16):7370–7389, 2014.

Tom B Brown. Language models are few-shot learners. *arXiv preprint ArXiv:2005.14165*, 2020.

Jiawei Chen, Hande Dong, Xiang Wang, Fuli Feng, Meng Wang, and Xiangnan He. Bias and debias in recommender system: A survey and future directions. *ACM Transactions on Information Systems*, 41(3):1–39, 2023.

Xi Chen, Yan Duan, Rein Houthooft, John Schulman, Ilya Sutskever, and Pieter Abbeel. Infogan: Interpretable representation learning by information maximizing generative adversarial nets. *Advances in neural information processing systems*, 29, 2016.

Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*, pages 191–198, 2016.

Jacob Devlin. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N Dauphin. Convolutional sequence to sequence learning. In *International conference on machine learning*, pages 1243–1252. PMLR, 2017.

Daniel Gillick, Alessandro Presta, and Gaurav Singh Tomar. End-to-end retrieval in continuous space. *arXiv preprint arXiv:1811.08008*, 2018.

Ian Goodfellow. *Deep learning*. MIT press, 2016.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*, pages 173–182, 2017.

Matthew Henderson, Iñigo Casanueva, Nikola Mrkšić, Pei-Hao Su, Tsung-Hsien Wen, and Ivan Vulić. Convert: Efficient and accurate conversational representations from transformers. *arXiv preprint arXiv:1911.03688*, 2019.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

Samuel Humeau, Kurt Shuster, Marie-Anne Lachaux, and Jason Weston. Poly-encoders: Transformer architectures and pre-training strategies for fast and accurate multi-sentence scoring. *arXiv preprint arXiv:1905.01969*, 2019.

Dietmar Jannach, Markus Zanker, Alexander Felfernig, and Gerhard Friedrich. *Recommender systems: an introduction*. Cambridge University Press, 2010.

Thorsten Joachims and Filip Radlinski. Search engines that learn from implicit feedback. *Computer*, 40(8):34–40, 2007.

Mozhgan Karimi, Dietmar Jannach, and Michael Jugovac. News recommender systems–survey and roads ahead. *Information Processing & Management*, 54(6):1203–1227, 2018.

Yoon Kim. Convolutional neural networks for sentence classification. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1746–

1751, Doha, Qatar, October 2014. Association for Computational Linguistics. doi: 10.3115/v1/D14-1181. URL https://aclanthology.org/D14-1181.

Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.

Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553): 436–444, 2015.

David Levy, Nic Newman, Richard Fletcher, Antonis Kalogeropoulos, and Rasmus Kleis Nielsen. Reuters institute digital news report 2014. *Report of the Reuters Institute for the Study of Journalism*, 2017.

Pasquale Lops, Marco De Gemmis, and Giovanni Semeraro. Content-based recommender systems: State of the art and trends. *Recommender systems handbook*, pages 73–105, 2011.

Guangyuan Ma, Hongtao Liu, Xing Wu, Wanhui Qian, Zhepeng Lv, Qing Yang, and Songlin Hu. Punr: Pre-training with user behavior modeling for news recommendation. *arXiv preprint arXiv:2304.12633*, 2023.

Jianxin Ma, Chang Zhou, Peng Cui, Hongxia Yang, and Wenwu Zhu. Learning disentan-

gled representations for recommendation. *Advances in neural information processing systems*, 32, 2019.

Prem Melville, Raymond J Mooney, Ramadass Nagarajan, et al. Content-boosted collaborative filtering for improved recommendations. *Aaai/iaai*, 23:187–192, 2002.

Tomas Mikolov. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

Lucas Möller and Sebastian Padó. Explaining neural news recommendation with attributions onto reading histories. *ACM Transactions on Intelligent Systems and Technology*, 2024.

Deuk Hee Park, Hyea Kyeong Kim, Il Young Choi, and Jae Kyeong Kim. A literature review and classification of recommender systems research. *Expert systems with applications*, 39(11):10059–10072, 2012.

Michael J Pazzani and Daniel Billsus. Content-based recommendation systems. In *The adaptive web: methods and strategies of web personalization*, pages 325–341. Springer, 2007.

Tao Qi, Fangzhao Wu, Chuhan Wu, and Yongfeng Huang. News recommendation with candidate-aware user modeling. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*, pages 1917–1921, 2022.

Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.

Shaina Raza and Chen Ding. News recommender system: a review of recent progress, challenges, and opportunities. *Artificial Intelligence Review*, pages 1–52, 2022.

David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*, pages 285–295, 2001.

J Ben Schafer, Dan Frankowski, Jon Herlocker, and Shilad Sen. Collaborative filtering recommender systems. In *The adaptive web: methods and strategies of web personalization*, pages 291–324. Springer, 2007.

Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural networks*, 61:85–117, 2015.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.

Harald Steck. Item popularity and recommendation accuracy. In *Proceedings of the fifth ACM conference on Recommender systems*, pages 125–132, 2011.

Yu Sun, Shuohuan Wang, Yukun Li, Shikun Feng, Xuyi Chen, Han Zhang, Xin Tian, Danxiang Zhu, Hao Tian, and Hua Wu. Ernie: Enhanced representation through knowledge integration. *arXiv preprint arXiv:1904.09223*, 2019.

Mian Muhammad Talha, Hikmat Ullah Khan, Saqib Iqbal, Mohammed Alghobiri, Tassawar Iqbal, and Muhammad Fayyaz. Deep learning in news recommender systems: A comprehensive survey, challenges and future trends. *Neurocomputing*, page 126881, 2023.

Ashish Vaswani. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.

Jingkun Wang, Yongtao Jiang, Haochen Li, and Wen Zhao. Improving news recommendation with channel-wise dynamic representations and contrastive user modeling. In *Proceedings of the sixteenth ACM international conference on web search and data mining*, pages 562–570, 2023.

Rongyao Wang, Shoujin Wang, Wenpeng Lu, and Xueping Peng. News recommendation via multi-interest news sequence modelling. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7942–7946. IEEE, 2022.

Chuhan Wu, Fangzhao Wu, Mingxiao An, Jianqiang Huang, Yongfeng Huang, and Xing Xie. Neural news recommendation with attentive multi-view learning. *arXiv preprint arXiv:1907.05576*, 2019a.

Chuhan Wu, Fangzhao Wu, Mingxiao An, Jianqiang Huang, Yongfeng Huang, and Xing

Xie. Npa: neural news recommendation with personalized attention. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2576–2584, 2019b.

Chuhan Wu, Fangzhao Wu, Suyu Ge, Tao Qi, Yongfeng Huang, and Xing Xie. Neural news recommendation with multi-head self-attention. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 6389–6394, 2019c.

Fangzhao Wu, Ying Qiao, Jiun-Hung Chen, Chuhan Wu, Tao Qi, Jianxun Lian, Danyang Liu, Xing Xie, Jianfeng Gao, Winnie Wu, et al. Mind: A large-scale dataset for news recommendation. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 3597–3606, 2020.

Haojie ZHANG, SUN Hui, QI Baiwen, and SHEN Zhidong. News recommendation system based on topic embedding and knowledge embedding. *Wuhan University Journal of Natural Sciences*, 28(1):29–34, 2023.